Applied Soft Computing Journal 187 (2026) 114314



Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc





HMP: Efficient heuristic algorithms for MDL-based itemset mining

Enze Chen [a], M. Zohaib Nawaz [a,b], M. Saqib Nawaz [a], Philippe Fournier-Viger [a,*]

[a] College of Computer Science and Software Engineering, Shenzhen University, 3688 Nanhai Avenue, 518060, Shenzhen, China
[b] Faculty of Computing and Information Technology, Department of Computer Science, University of Sargodha, Chaudhry Muhammad Ali Block, University road, 40100, Sargodha, Pakistan

HIGHLIGHTS

- Introduces HMP, a framework combining Simulated Annealing and Hill Climbing with the Minimum Description Length principle for efficient pattern selection.
- HMP enhances computational efficiency and pattern quality, reducing runtime while ensuring representative itemsets.
- Demonstrates superior performance over GRIMP, KRIMP, and SLIM in runtime and compression ratio, as well as excellent downstream classification accuracy.

ARTICLE INFO

Keywords:
Frequent itemset mining
Minimum description length
Simulated annealing
Hill climbing
Heuristic algorithms

ABSTRACT

Frequent itemset mining is a critical task for discovering interesting patterns in large datasets, yet traditional algorithms often overwhelm users with large pattern sets that hinder interpretation. Consequently, recent research has focused on discovering compact and informative pattern sets based on the Minimum Description Length (MDL) principle. Despite their effectiveness, existing MDL-based algorithms still suffer from long runtimes and do not guarantee optimal compression. To address these challenges, this paper introduces the novel HMP framework (Heuristic MDL-based Pattern mining), which supports two alternative heuristic algorithms for MDL-based pattern selection. The first, HMP-SA, adopts a Simulated Annealing (SA)-based optimization process to explore the search space more efficiently, reducing computational time while producing more representative itemsets, while the second, HMP-HC, utilizes a Hill Climbing (HC) approach. Both algorithms adopt a partial coverage function that enables partial and heuristic MDL-based compression, and rely on a binary matrix optimization to efficiently generate and evaluate item pairs. Extensive experimental evaluations demonstrate that HMP-SA and HMP-HC outperform the state-of-the-art GRIMP algorithm as well as the earlier KRIMP and SLIM algorithms in terms of both runtime and compression ratio, with HMP-SA also demonstrating strong downstream classification performance.

1. Introduction

Discovering hidden patterns in large datasets is the fundamental goal of data mining. A key task in this domain is Frequent Itemset Mining (FIM) [2,25], which identifies sets of values that frequently co-occur within a dataset. The task of FIM is traditionally formulated as follows. Consider a dataset containing multiple transactions (records), where each transaction consists of a set of items (values). Given a minimum support (*min_sup*) threshold, FIM entails enumerating all the frequent itemsets. A frequent itemset is a set of items that appear together in at least *min_sup* transactions. The extraction of frequent itemsets reveals strong associations between items, providing valuable insights into underlying patterns—for instance, identifying groups of co-occurring words in text documents or common symptoms among patients in healthcare settings. Moreover, frequent itemsets are also useful in several downstream data mining or machine learning tasks, including classification and clustering [1].

Despite its utility, FIM algorithms often produce an overwhelming number of patterns. Even in modestly sized datasets, it is not uncommon to discover millions of frequent itemsets, many of which are highly similar to one another. This issue, known as the *pattern explosion problem* [41], can significantly complicate the process of extracting meaningful insights and hinder the application of FIM in real-world scenarios. Various strategies have been proposed to reduce the number of patterns generated by FIM algorithms. Most of these approaches aim to find specific subsets of all frequent itemsets that satisfy additional criteria

* Corresponding author.
Email address: philfv@szu.edu.cn (P. Fournier-Viger).

https://doi.org/10.1016/j.asoc.2025.114314
Received 16 July 2025; Received in revised form 4 November 2025; Accepted 20 November 2025
Available online 21 November 2025
1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

such as maximal itemsets [28], closed itemsets [34,40], non-derivable itemsets [10], free-sets [8], statistically significant itemsets [12], local dependency itemsets [33], and other itemsets that meet various constraints [5,7,22]. However, even these refined approaches may still overwhelm users with large, redundant sets of patterns because the decision to output each pattern is made independently of the decisions to output other patterns. Consequently, many itemsets appear in the output that cover the same transactions and differ only slightly from one another, causing redundancy. For example, in a retail dataset, the frequent itemsets '*egg, muffin*', '*egg, orange juice*', and '*egg, muffin, orange juice*' might all be identified as frequent and have similar frequencies, leading to redundancy and making it difficult to prioritize truly valuable patterns.

A prominent solution to both pattern explosion and redundancy is to employ the Minimum Description Length (MDL) [13] principle to select a concise and representative set of itemsets. This approach, introduced in the KRIMP algorithm [41], searches for the model (set of itemsets) that most effectively encodes the database in accordance with the MDL. The core idea is that the *model which compresses the database most effectively also best describes it*, capturing essential patterns while reducing redundancy. Focusing on compression, the MDL principle facilitates the identification of a compact and informative representation of the data. The optimal model is the one that attains the minimal total description length, defined as the sum of the model's size and the size of the database when encoded with the model. This ensures that the itemsets are representative, diverse, and meaningful. However, identifying the optimal set of itemsets based on the MDL principle is more challenging than the traditional FIM problem. In FIM, the number of potential itemsets from n distinct items is 2^n-1. In contrast, MDL-based itemset mining requires selecting the most optimal subset of these itemsets to construct a compact model. This selection process results in an exponentially larger search space of size $2^{(2^n-1)}$. This exponential increase in the search space underscores the computational intractability of the MDL-based approach. Hence, a key challenge is to develop efficient algorithms that can find a satisfactory solution without evaluating all possible solutions.

KRIMP [41] and SLIM [37], the two most representative MDL-based itemset mining algorithms, employ a greedy strategy that iteratively constructs a set of itemsets by sequentially adding one pattern at a time until further compression improvements are no longer attainable. While this method can identify good sets of patterns, it remains computationally intensive. Moreover due to the narrow focus of the search—prioritizing stepwise local improvements—these algorithms fail to explore the search space broadly, potentially overlooking globally optimal solutions. To achieve better performance and find a more optimal set of patterns in terms of MDL-based compression, a recent algorithm called GRIMP [32] was introduced, based on genetic algorithm (GA) principles [17,31]. To enable the use of heuristic search in MDL-based itemset mining, GRIMP introduced a partial coverage function that relaxes the strict requirement for itemsets to fully cover the database during compression. While GRIMP demonstrated improved speed and compression compared to SLIM and KRIMP, there is still considerable potential for further advancements in both computational efficiency and pattern quality.

In this work, we aim to address the limitations of existing MDL-based methods for discovering representative patterns by proposing a generic framework, called HMP (Heuristic MDL-based Pattern mining). HMP is designed to discover itemsets that are representative according to the MDL principle using heuristics. The framework offers two heuristic algorithms, HMP-SA, based on Simulated Annealing (SA) [6], and HMP-HC, based on Hill Climbing (HC) [36], to efficiently discover near-optimal and diverse sets of itemsets, thereby improving the quality and representativeness of the extracted patterns. The key contributions of this paper are as follows:

- A generic framework called HMP is presented for heuristic MDL-based pattern mining. The framework incorporates two heuristic algorithms, named HMP-SA and HMP-HC, which respectively adopt a SA [6] and a HC [36] based approach. The framework adopts the partial coverage function of GRIMP [32], which relaxes the strict full-database coverage constraint in traditional MDL-based compression.
- A binary co-occurrence matrix is introduced in HMP to efficiently evaluate all item pairs. This matrix is constructed in a single dataset pass and stores the support of every distinct item pair in the dataset, allowing constant-time support retrieval without repeatedly scanning the dataset. By maintaining pairwise supports in memory, this structure enables rapid heuristic evaluation of item pairs for compression. Consequently, the binary matrix optimization substantially improves runtime efficiency, and accelerates convergence.
- A custom classifier based on HMP is designed to utilize the pattern found for classification, where each class is represented by a code table, and unseen transactions are classified based on the code table that provides the most concise encoding.
- An extensive experimental evaluation demonstrates that HMP-SA and HMP-HC outperform the state-of-the-art algorithms GRIMP, KRIMP, and SLIM in terms of runtime efficiency and compression ratios, while achieving competitive performance in the downstream classification task.

The remainder of this paper is organized as follows. Section 2 surveys related work on MDL-based pattern mining and heuristic algorithms. Section 3 provides the necessary background and problem formulation. Section 4 presents the HMP framework, along with the HMP-SA and HMP-HC algorithms. Section 5 outlines the experimental setup and results. Finally, Section 6 summarizes the contributions and discusses directions for future work.

2. Related work

In this section, a review of MDL-based pattern mining approaches is first presented, followed by a discussion of heuristic methods—specifically SA and HC—and their applications in itemset mining.

2.1. Pattern mining based on the MDL principle

Vreeken et al. [41] introduced the MDL-based approach for pattern mining in the KRIMP algorithm. The goal is to extract the set of itemsets that best compress a dataset. KRIMP takes as input a transaction dataset and a set of frequent itemsets, which are extracted in advance by a traditional FIM algorithm. Subsequently, KRIMP searches for the subset of frequent itemsets that minimizes the overall description length. This search process is iterative; at each iteration, KRIMP selects the itemset that most effectively reduces the total description length. The search terminates when no further improvements can be made. It has been demonstrated that this approach can identify sets of itemsets that provide high compression. However, KRIMP has three main limitations: (1) the quality of the final results is heavily dependent on the frequent itemsets extracted in advance, which relies on the *min_sup* threshold set by the user, (2) the extraction of frequent itemsets in advance is a computationally intensive and time-consuming process, and (3) the search process constructs the solution in a fixed order, one itemset at a time, without backtracking, which can lead to suboptimal solutions.

To address the first two problems, the SLIM algorithm [37] was developed. A key improvement is that SLIM does not require mining frequent itemsets in advance. Instead, SLIM generates frequent itemsets incrementally as it constructs a solution. This approach is considerably faster and eliminates the dependency on the *min_sup* threshold. However, the solutions are still constructed incrementally without backtracking, and hence an optimal solution may not be identified if certain itemsets are not selected early in the search process. Some studies have also focused on accelerating KRIMP. For instance, the SHRIMP algorithm [16] utilizes a prefix-tree structure to rapidly assess the impact of a pattern on compression. Similarly, another variant of KRIMP, named RSCO

[35], applies techniques for parallel computation to reduce runtimes. However, despite these improvements, runtimes remain a significant concern for algorithms based on KRIMP. Several variations of the MDL-based approach for FIM have also been introduced. For characterizing the similarities and differences among multiple transaction datasets, SLIM was adapted into the DIFFNORM algorithm [9]. Additionally, the PACK algorithm [38] was developed to compress a dataset by selecting itemsets using decision trees and a refined version of the MDL. This approach was shown to generate good compression but can produce large sets of itemsets.

The MDL principle has also been applied in other pattern mining tasks beyond itemset mining to analyze various data types, such as graphs, sequences, numerical data, and text data. For graph data, the CSPM algorithm [24] was introduced to extract star-shaped patterns that compress large attributed graphs based on the MDL principle. This algorithm has been successfully applied to the analysis of alarm occurrences in telecommunication network data, and for graph attribute completion. For sequence data, KRIMP was extended as the SEQKRIMP and GOKRIMP algorithms [20] for mining sequential patterns that compress sequences of symbols. These algorithms have been applied to several datasets, including journal abstracts (sequences of words). While SEQKRIMP adopts a two-phase approach for pattern mining, GOKRIMP is a greedy algorithm that incorporates a dependency test to identify high quality patterns and decrease the runtime. To analyze streams, the SWIFT algorithm [43] uses the MDL principle to continuously update a compact set of representative patterns that maximize data compression in event streams. A version called B-SWIFT was also introduced in the same paper for batch processing. For numerical data, an extended version of KRIMP called REALKRIMP [42] was put forward, which processes real-valued data via discretization and various heuristics. Another algorithm for mining patterns in numerical data based on the MDL principle is MINT [26], which can identify a compact set of non-redundant and informative patterns. For text data, the DRIM algorithm [39] extends the SLIM method by applying the MDL principle to text summarization, using a transactional dataset representation and a diversity constraint to generate concise and non-redundant summaries. To find recurring spatial structures in discrete, geometric matrices such as raster data, the VOUW algorithm [11] uses the MDL principle to select a set of patterns that capture complex spatial relations, resulting in arbitrarily shaped patterns. This approach demonstrated high-quality results on synthetic data.

Patterns found based on the MDL have also been used in various applications such as for classification [37] and anomaly detection [4]. For instance, to identify anomalies in large multi-dimensional databases, the COMPREX algorithm [4] employs pattern-based compression to generate a collection of dictionaries that succinctly describe the norm of a database, flagging points with high compression costs as anomalies. The problem of anomaly detection using patterns extracted based on the MDL principle was also studied for financial records [23], where significant patterns were discovered in real data. Some studies have also used the MDL in other ways. An algorithm [45] was designed to leverage the MDL principle in graph-structured data to efficiently discover meaningful patterns in scenarios where samples are scarce and come from different entities with inherent structural relationships. By representing entities as a graph and integrating sample weights using kernel functions into an MDL-based filtering process, the method generates differentiated pattern sets for each node. In another work, the SCIM algorithm [27] was designed based on dual scaling to map items from a database to a multidimensional space, to then retrieve representative closed itemsets based on a distance threshold and using tree structures.

Despite significant progress in MDL-based pattern mining, existing algorithms still face challenges due to the vast search space, often resulting in long runtimes and sub-optimal solutions. A promising direction to address these limitations is the use of heuristic algorithms that could offer a better balance between exploration and exploitation of different parts of the search space. One such recent approach is GRIMP [32], which integrates GAs with the MDL principle to improve pattern mining efficiency. GRIMP investigates different GA variants and evaluates their effectiveness across various datasets. GRIMP not only surpasses KRIMP and SLIM in terms of compression ratio and runtime but also discovers pattern sets that enhance downstream classification tasks. Table 1 provides a summary of the main approaches for MDL-based pattern mining discussed in this subsection.

2.2. Simulated annealing and hill climbing

Heuristic methods are widely utilized in optimization problems to identify high-quality solutions within a reasonable time frame, particularly when exact approaches are computationally prohibitive or impractical. Heuristics trade off optimality for speed, making them especially effective for addressing complex, large-scale problems where finding an exact solution is either infeasible or excessively time-consuming. Numerous heuristic and metaheuristic approaches have been proposed in the literature, including GA, SA, HC, Bat Algorithm (BA), Ant Colony Optimization (ACO), and Particle Swarm Optimization (PSO) [3].

Table 1
A summary of the main approaches for MDL-based pattern mining.

Category	Key Algorithms	Key Features	Limitations/Notes
MDL-based itemset mining	KRIMP [41]	Uses pre-mined frequent itemsets to iteratively select patterns that minimize description length.	Dependent on user-defined *min_sup*, computationally intensive, and employs a fixed, non-backtracking search.
	SLIM [37]	Generates frequent itemsets incrementally during solution construction, eliminating the pre-mining step.	Still uses an incremental, no-backtracking approach, potentially leading to suboptimal solutions.
	SHRIMP [16], RSCO [35]	Incorporate prefix-tree structures and parallel computing to assess pattern impact and reduce runtime.	Despite improvements, runtimes remain significant due to the large search space.
Extensions to other data types	CSPM [24], SEQKRIMP [20], GOKRIMP [20], SWIFT [43], REALKRIMP [42], MINT [26], DRIM [39], VOUW [11]	Adapt the MDL principle to graph, sequential, numerical, text, and spatial data, each tailored to specific data characteristics.	Some approaches may produce large or redundant pattern sets and are optimized for particular data types.
Other variants	DIFFNORM [9], PACK [38], SCIM [27]	Include techniques like decision trees, dual scaling, and cross-dataset analysis to refine pattern selection.	Present varied trade-offs between compression quality and computational complexity.
Heuristic-based algorithms	GRIMP [32]	Designed for MDL-based pattern mining, shows promising results compared to traditional algorithms like KRIMP and SLIM.	Can have relatively long runtimes and there is still room for improvement in the quality of generated solutions.
	HMP (proposed framework)	SA- and HC-based approaches to balance exploration and exploitation in the search space.	Further improvements could be done in future work.

SA [6,30] is a probabilistic optimization technique inspired by the physical annealing of metals, in which a material is heated to a high temperature and cooled slowly to reduce structural defects. Similarly, SA searches large solution spaces by initially accepting solutions that provide both improvements and degradations with relatively high probability, then progressively focuses on higher-quality solutions to escape local optima. This process follows a geometric cooling schedule, in which the temperature at an iteration $t+1$ is derived from the temperature at the previous iteration t by the equation:

$$T_{t+1} = \alpha T_t \tag{1}$$

where $\alpha \in (0,1)$ denotes the cooling rate. The algorithm relies on three hyperparameters: the initial temperature T_{init}, which controls the breadth of early exploration; the cooling rate α, which determines the speed of temperature decay; and the minimum temperature T_{min}, at which the algorithm terminates. At each iteration, a candidate solution is accepted in place of the current solution according to:

$$P(\text{accept}) = \begin{cases} 1, & \Delta S \geq 0, \\ e^{\frac{\Delta S}{T}}, & \Delta S < 0 \end{cases} \tag{2}$$

where $\Delta S = S' - S$ is the difference between the objective value of the new solution S' and that of the current best solution S. As T decreases, the likelihood of accepting inferior solutions diminishes, thereby balancing global exploration and local exploitation.

HC [36] is a simpler heuristic optimization algorithm that iteratively refines a candidate solution with regard to a fitness function. At each step, the method examines the immediate neighborhood of the current solution and transitions to the neighboring solution with the highest improvement, if one exists. Unlike SA, which can accept non-improving solutions to escape local optima through a probabilistic cooling mechanism, HC strictly accepts only better solutions. This difference allows HC to converge rapidly, but also makes it more prone to becoming trapped in local optima within complex search spaces, as it lacks the ability to explore less favorable regions.

For pattern mining, many heuristics have been applied to a variety of problems with different degrees of success [14]. In the context of MDL-based pattern mining, to our knowledge, only GAs have been investigated in the GRIMP [32] algorithm. Although GRIMP has shown promising improvements over traditional MDL-based methods such as KRIMP and SLIM, it still incurs relatively long runtimes and the quality of its solutions can still be improved.

In this paper, we aim to enhance both the efficiency and the solution quality of MDL-based itemset mining. We propose the HMP framework, which comprises two novel algorithms based on SA and HC. The reasons for selecting SA and HC are twofold. First, while several heuristics have multiple parameters, SA and HC have few parameters, which makes them user-friendly, while still being very efficient and effective. Second, we are motivated by a recent study [29] that has shown that SA and HC can yield excellent results for other pattern mining problems such as high utility itemset mining when compared to other heuristics such as PSO, BA, and GA.

The next section presents the background and problem formulation for FIM and MDL-based pattern mining using heuristics. Then, the proposed HMP framework based on SA and HC is described in Section 4.

3. Background and problem formulation

This section is divided into three parts. It begins by introducing the traditional task of FIM, followed by a description of itemset mining using the MDL principle. Finally, it presents the formulation of the MDL-based itemset mining problem for heuristic algorithms.

3.1. Mining frequent itemsets

The task of FIM [2] can be defined as follows: Let $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ be a set of distinct items (values). A transaction database $\mathcal{D} = \{t_1, t_2, \ldots, t_n\}$ is a set of n records, called transactions, where each transaction $t \in \mathcal{D}$ is a non-empty subset of items from $\mathcal{I}$, i.e., $t \neq \emptyset$ and $t \subseteq \mathcal{I}$. An itemset $\mathcal{X}$ is a subset $\mathcal{X} \subseteq \mathcal{I}$, and it is said to be a singleton itemset if it contains one item, i.e. its cardinality is $|\mathcal{X}| = 1$. In the following, the notation $P(\mathcal{I})$ is used to denote the powerset of $\mathcal{I}$, which is the set of all subsets of $\mathcal{I}$.

Table 2
An example transaction database.

Transactions	Items
t_1	a, c, d, e, f
t_2	d, e, f
t_3	a, c, e
t_4	a, f
t_5	b, f

The support of an itemset $\mathcal{X}$ in a database $\mathcal{D}$ is defined as the number of transactions in $\mathcal{D}$ that include $\mathcal{X}$, and it is denoted as $sup(\mathcal{X})$. A transaction $t \in \mathcal{D}$ supports an itemset $\mathcal{X}$ if all items of $\mathcal{X}$ are included in t, i.e. $\mathcal{X} \subseteq t$. The goal of FIM is to list all frequent itemsets from the input database $\mathcal{D}$. An itemset is considered frequent if its support is at least equal to a user-defined threshold called *minimum support* (min_sup), i.e., $sup(\mathcal{X}) \geq min_sup$.

For example, Table 2 shows a transaction database containing five transactions and six different items represented by the letters a, b, c, d, e and f. If $\min_sup = 2$, the frequent itemsets are: $\{a\} : 3$, $\{c\} : 2$, $\{d\} : 2$, $\{e\} : 3$, $\{f\} : 4$, $\{a, c\} : 2$, $\{a, e\} : 2$, $\{c, e\} : 2$, $\{d, e\} : 2$, $\{a, f\} : 2$, $\{d, f\} : 2$, $\{e, f\} : 2$, $\{a, c, e\} : 2$, and $\{d, e, f\} : 2$, where the number after each itemset indicates its support. For instance, itemset a, c appears in transactions t_1 and t_3, and hence its support is 2.

3.2. Mining itemsets based on the MDL principle

Although traditional FIM algorithms have many applications, the quality of their results largely depends on how the min_sup threshold is set by the user, and these algorithms can produce numerous redundant itemsets that overlap with each other [37,41]. Algorithms for MDL-based itemset mining address these issues by adopting a different approach. They analyze an input database D to identify the most efficient model $\mathcal{M}$ for compressing it. In this context, a model $\mathcal{M}$ consists of multiple itemsets, and the database is compressed using the model by replacing each occurrence of an itemset from the model with a corresponding code. MDL-based itemset mining algorithms explore multiple candidate models, evaluating each with the goal of finding the one that yields the best compression. The suitability of a model $\mathcal{M}$ for compressing a database $\mathcal{D}$ under the MDL principle is evaluated by Eq. (3):

$$L(\mathcal{D}, \mathcal{M}) = L(\mathcal{D}|\mathcal{M}) + L(\mathcal{M}) \tag{3}$$

Here, the term $L(\mathcal{M})$ denotes the description length of the model in bits, and $L(\mathcal{D}|\mathcal{M})$ represents the number of bits required to encode the database $\mathcal{D}$ using $\mathcal{M}$. A more effective model is one that minimizes $L(\mathcal{D}, \mathcal{M})$, indicating a smaller combined size for the encoded data and the model itself.

For MDL-based itemset mining, each model $\mathcal{M}$ is represented by a structure named the *code table* [37,41]. A code table C is a table with two columns, *Itemsets* and *Codes*. The *Itemsets* column stores all itemsets from the model, while the *Codes* column specifies the unique code assigned to each corresponding itemset for compressing the database. A row from a code table is represented as a tuple $(\mathcal{X}, code(\mathcal{X}))$ where $\mathcal{X} \subseteq \mathcal{I}$ is an itemset and $code(\mathcal{X})$ is its unique code. The set of all codes in a C is denoted by $codeset(C)$ and called its coding set. A code table that contains exactly the itemsets from a given set Z (and no others) is denoted by C_Z.

For instance, consider the database in Table 2, where the set of items is $\mathcal{I} = \{a, b, c, d, e, f\}$. A possible code table for compressing this database is shown in Table 3. It contains six itemsets, namely $\{a, c, e\}$, $\{d, f\}$,

Table 3
A code table (model) for compressing the database shown in Table 2. Codes are represented by colored rectangles, with each color corresponding to a distinct code. The width of a rectangle indicates the relative length of the code.

Itemsets	Codes	Usage
$\{a, c, e\}$		2
$\{d, f\}$		2
$\{a, f\}$		1
$\{b\}$		1
$\{e\}$		1
$\{f\}$		1

Table 4
An example database (left) and its encoding using the code table of Table 3 (right).

Database	Encoded database
$\{a, c, e\}$ $\{d, f\}$	
$\{d, f\}$ $\{e\}$	
$\{a, c, e\}$	
$\{a, f\}$	
$\{b\}$ $\{f\}$	

$\{a, f\}$, $\{b\}$, $\{e\}$, and $\{f\}$. In the *Codes* column, each code is depicted as a colored rectangle whose width reflects its relative length (a longer rectangle indicates a longer code). A third column, *Usage*, is also added for explanatory purposes. It indicates how many times an itemset is used for compressing the database. The *Usage* column is included for this example, but is not an actual component of code tables. Generally, every itemset in a code table has a usage of at least 1.

The compression of a database D with a code table C is achieved by replacing each occurrence of an itemset in D with its corresponding code in C. This process requires first selecting a cover function:

$$cover: D \rightarrow 2^{C_Z} \tag{4}$$

which assigns to each transaction $t \in D$ a subset $cover(t)$ of itemsets such that: (1) the union of all itemsets in $cover(t)$ exactly matches the items in t, i.e. $\bigcup_{\mathcal{X} \in cover(t)} code(\mathcal{X}) = t$, and (2) no itemset in $cover(t)$ is strictly contained within another, i.e. $\forall \mathcal{X}, \mathcal{Y} \in cover(t), \mathcal{X} \neq \mathcal{Y} \Rightarrow \mathcal{X} \not\subset \mathcal{Y}$. Once a cover function is determined, the database can be compressed. For each transaction t, each itemset $\mathcal{X} \in cover(t)$ is replaced by its corresponding code. Thus, the compressed representation of a transaction t is the code set $t' = \{ code(\mathcal{X}) : \mathcal{X} \in cover(t)\}$.

For example, Table 4 illustrates how the database of Table 2 can be encoded using the code table presented in Table 3. The left column of Table 4 shows the original database, while the right column shows the encoded database. In this example, the cover for the first transaction $t_1 = \{a, c, e, d, f\}$ is defined as $cover(t_1) = \{\{a, c, e\}, \{d, f\}\}$. After encoding, the first transaction is represented by the codes for $\{a, c, e\}$ (the green rectangle) and $\{d, f\}$ (the pink rectangle). Similarly, the remaining transactions are encoded using one or more itemsets from C. Transactions may be encoded by multiple itemsets, and any given itemset may encode multiple transactions.

Since there can be multiple ways to match itemsets from a code table C with a database D, the resulting compression depends on the chosen cover function, and more precisely on the order in which itemsets are matched with transactions. To achieve optimal compression, algorithms such as KRIMP and SLIM employ a total order called the *Standard Cover Order* [41]. This order ranks itemsets first by decreasing size, then by descending usage count, and finally by ascending lexicographical order. Intuitively, this means that a database is compressed with a code table C by first matching longer and more frequently occurring itemsets from C before those that are shorter or less common. The ordering is also used for assigning codes to itemsets; those ranked higher receive shorter codes, to increase compression. Itemsets in code tables are thus sorted according to this order in algorithms such as KRIMP [41] and SLIM [37].

With regard to codes given to itemsets, previous examples depicted codes as colored rectangles because only their lengths matter for MDL-based itemset mining. The actual codes are irrelevant because the goal is to use compression to extract itemsets and only the code lengths are needed to calculate the model size $L(\mathcal{M})$ and encoded database size $L(D|\mathcal{M})$ in bits.

Calculating optimal code lengths for itemsets from a code table C and a database D is done by computing the Shannon entropy of each itemset's usage frequency. The usage of an itemset $\mathcal{X}$ is the number of transactions in which $\mathcal{X}$ appears as part of the cover:

$$usage(\mathcal{X}) = |\{t \in D | \mathcal{X} \in cover(t)\}| \tag{5}$$

The optimal code length [21] of an itemset $\mathcal{X}$ for a code table is then calculated as:

$$L(\text{code}(\mathcal{X})) = -\log_2(usage(\mathcal{X})/|D|) \tag{6}$$

Note that in MDL-based itemset mining, fractional lengths are used for greater precision rather than rounding lengths to the nearest integer.

To illustrate code length calculations, consider the itemset $\{a, c, e\}$ from the code table of Table 3. For encoding the database depicted in Table 2, $\{a, c, e\}$ has a usage count of 2, it is in the cover of two transactions (t_1 and t_3). Since there are five transactions in total, the optimal code length of $\{a, c, e\}$ is $L(\text{code}(\{a, c, e\})) = -\log_2\left(\frac{2}{5}\right) = 1.32$ bits. The code lengths of other itemsets can be obtained in the same way. For example, $L(\text{code}(\{f\})) = -\log_2\left(\frac{1}{5}\right) = 2.32$ bits.

Then, how good a model C is for encoding a database D is calculated based on Eq. (3) as $L(D, C) = L(D|C) + L(C|D)$ such that:

$$L(D|C) = \sum_{t \in D} L(t|C) \tag{7}$$

$$L(t|C) = \sum_{X \in cover(t)} L(\mathcal{X}|C) \tag{8}$$

$$L(C|D) = \sum_{\substack{\mathcal{X} \in CT \\ usage(\mathcal{X}) \neq 0}} (L(\mathcal{X}|ST) + L(\mathcal{X}|C)) \tag{9}$$

where $L(D|C)$ is the encoded size of D (in bits) and $L(C|D)$ is the size of C. In Eq. (9), the term ST denotes the *Standard Code Table*, which is the code table that only includes the single items from the database. The problem of MDL-based pattern mining is formulated as follows [41].

Problem of MDL-based itemset mining: For a database D with items $\mathcal{I}$, the problem is to find a set of itemsets $Z \subseteq P(\mathcal{I})$ such that $L(D, C_Z)$ is minimal.

3.3. Mining itemsets based on the MDL principle with heuristics

The previous formulation of MDL-based itemset mining suits exact algorithms, where the objective is to identify a set of itemsets that fully covers every transaction in the database. Heuristic algorithms, however, cannot guarantee full coverage due to the probabilistic exploration of the search space [32]. Therefore, it is necessary to adjust the problem formulation to focus on finding a set of itemsets that provides high compression, but may not cover the entire database, leaving other parts of the database uncompressed.

The revised formulation of MDL-based itemset mining for heuristic algorithms is as follows. For a code table C and a database D, the total

encoded size is calculated as:

$$L_H(D,C) = L_H(D|C) + L_H(C|D) \tag{10}$$

where the term $L_H(D|C)$ is the size of the encoded database using C:

$$L_H(D|C) = \sum_{t \in D} L_H(t|C) \tag{11}$$

$$L_H(t|C) = \sum_{\mathcal{X} \in cover(t)} L(\mathcal{X}|C) + \sum_{i \in t \setminus \text{cover}(t)} L(i|C) \tag{12}$$

and the term $L_H(C|D)$ is the size of C, calculated as:

$$L_H(C|D) = \sum_{\substack{\mathcal{X} \in C \\ usage(\mathcal{X}) \neq 0}} (L(\mathcal{X}|C) + L(code(\mathcal{X}))) \tag{13}$$

In this formulation, the definition of the cover function (denoted as *cover*) is relaxed to allow partial coverage, i.e. not all items from a transaction t need to be matched. Thus, the function *cover* is selected such that for each transaction t, $\bigcup_{\mathcal{X} \in cover(t)} \mathcal{X} \subseteq t$ and $\forall \mathcal{X}, \mathcal{Y} \in cover(t), \mathcal{X} \neq \mathcal{Y} \Rightarrow \mathcal{X} \not\subset \mathcal{Y}$. The notation $t \setminus cover(t)$ refers to itemsets from t that are not matched by the cover function. Besides, the notation $L(\mathcal{X}|C)$ denotes the encoded length of an itemset $\mathcal{X}$ based on a code table C. The notation $L(i|C)$ is the encoded length of an item i that is not covered by the cover function.

The modified formulation of MDL-based itemset mining based on $L_H(C,D)$ is employed for the remainder of this paper. This approach is necessary for developing heuristic methods and allows a fair comparison with exact MDL-based approaches. More details about this revised formulation can be found in [32].

Furthermore, another modification to the traditional MDL-based pattern mining approach [37,41] adopted in this work is that the code table is not sorted; itemsets are inserted in the order in which they are discovered. This design choice simplifies the computation of database compression when a new itemset is added, as only the previously uncompressed portion of the database is affected by adding an itemset. In contrast, if the code table were sorted, the compression of the entire database may need to be recomputed from scratch after inserting an itemset.

4. The HMP framework

This section presents the proposed HMP framework for efficient MDL-based itemset mining using heuristic algorithms. This framework consists of two algorithms, namely HMP-SA and HMP-HC, which both follow the same general process for mining itemsets according to the MDL principle. This general process is illustrated in Fig. 1.

The input to the framework is a transaction database D, and the output is a set of itemsets that yield high compression of the database according to the MDL principle. The process begins by scanning the transaction database to determine its original size. Next, an empty code table C is created for storing the discovered itemsets. Following this, a candidate itemset is generated randomly. A number of iterations are then performed to improve this itemset. This iterative process consists of several steps, represented by purple boxes in Fig. 1. During each iteration, a random modification is applied to the candidate itemset using a function called `GetNeighbor`, which adds or removes an item. The modified candidate itemset is then evaluated to determine if adding it to the current C would improve compression or if another acceptance criterion is met. If the modification is not beneficial, it is reverted; otherwise, it is retained. This iterative improvement continues until a maximum number of iterations is reached. After that, the candidate itemset is added to the code table C, and the transaction database is updated accordingly. The framework continues to generate and refine new candidate itemsets until the size of C reaches a maximum size. Finally, to further enhance compression, HMP adds the most frequent itemsets of length 2 to C. Once this process is complete, the code table C, now containing a set of itemsets optimized for database compression, is returned to the user. The proposed HMP algorithm applies heuristics to explore the search space of itemsets and only discard itemsets if adding them to the code table does not improve database compression.

Before explaining the details of the HMP-SA and HMP-HC algorithms, the following Section 4.1 describes key concepts shared by both algorithms. Then, Sections 4.2 and 4.3 describe the HMP-SA and HMP-HC algorithms, respectively.

4.1. Common concepts and structures

This section introduces two important concepts used by both the HMP-SA and HMP-HC algorithms. The section starts by describing the `GetNeighbor` function, which is utilized to modify a candidate itemset by adding or removing items, so as to explore neighboring solutions that may provide higher compression. Then, the section introduces a matrix structure that is utilized to count and store the support of all itemsets of size two in the database. This matrix is used by both algorithms to quickly find itemsets of size two that could be added to a code table to further improve the compression.

4.1.1. The get neighbor function

A core operation in HMP is to modify an itemset to improve its compression potential. This process consists of adding or removing an item from the itemset to explore its neighborhood. This operation is implemented by the `GetNeighbor` function (Algorithm 1), which takes as input

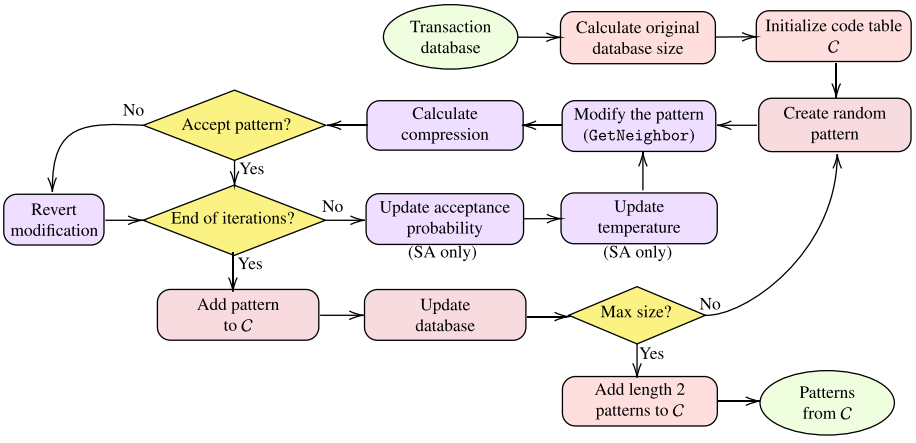


Fig. 1. The overall flowchart of the proposed HMP framework for heuristic MDL-based pattern mining, including two heuristics (HMP-SA and HMP-HC). The iterative process to improve a pattern consists of several steps represented by purple boxes. Steps specific to HMP-SA are indicated as "(SA only)".

Algorithm 1 GetNeighbor.

Input: An itemset p such that $m_{\min} \leq |p| \leq m_{\max}$, the set of all items with weights $\mathcal{I}_w = \{(i_1, w_1), \ldots, (i_n, w_n)\}$, minimum and maximum pattern sizes $m_{\min}$ and $m_{\max}$
Output: A modified itemset p' such that $m_{\min} \leq |p'| \leq m_{\max}$

if $|p| = m_{\min}$ **then**
  Select an item $i \notin p$ from $\mathcal{I}_w$ with probability proportional to w_i
else if $|p| = m_{\max}$ **then**
  Select an item $i \in p$ from $\mathcal{I}_w$ with probability proportional to w_i
else
  Select any item i from $\mathcal{I}_w$ with probability proportional to w_i
end if
if $i \in p$ **then**
  $p' \leftarrow p \setminus \{i\}$
else
  $p' \leftarrow p \cup \{i\}$
end if
return p'

an itemset p, representing the current pattern being considered, a set $\mathcal{I}_w$ containing all distinct items from the database along with weights indicating their estimated importance for compression, and two integers $m_{\min}$ and $m_{\max}$ defining the allowed minimum and maximum sizes of patterns. The output is a modified itemset p' that differs from p by exactly one item (either added or removed) and satisfies the size constraint $m_{\min} \leq |p'| \leq m_{\max}$.

The `GetNeighbor` function begins by assessing the cardinality of the current pattern p. If $|p| = m_{\min}$, the pattern is expanded by selecting and adding an item not in p, drawn with probability proportional to its weight. If $|p| = m_{\max}$, the pattern size is reduced by selecting and removing an item from p, again with probability proportional to its weight. Otherwise, the procedure chooses any item and either adds it (if it is absent) or removes it (if it is present), thereby producing a neighbor p' that differs from p by exactly one item while always respecting the size constraints.

The weights of items are calculated as follows. Consider that the input database D contains m items, that is $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$. The set $\mathcal{I}_w$ is formally defined as:

$$\mathcal{I}_w = \{(i_k, w_k) | i_k \in \mathcal{I}\} \tag{14}$$

where the weight of item i_k $(1 \leq k \leq m)$ is defined as its relative frequency in the database:

$$w_k = \frac{\sup(\{i_k\})}{|D|} \tag{15}$$

Thus, items that appear more frequently are assigned higher weights, since they are more likely to contribute significantly to compression.

To illustrate the calculation of weights, take the database of Table 2 as an example. The total number of transactions is $|D| = 5$. The items a, b, c, d, e, and f respectively have supports of 3, 1, 2, 2, 3, and 4. Thus, the set $\mathcal{I}_w$ of items with their weights is:

$$\mathcal{I}_w = \{(a, 0.6), (b, 0.2), (c, 0.4), (d, 0.4), (e, 0.6), (f, 0.8)\}$$

Based on these weights, the `GetNeighbor` function is most likely to select item f, followed by a and e, then c and d, and finally b.

The `GetNeighbor` function is simple and efficient. Adding or removing an item systematically leads HMP to examine a wide range of possible solutions, thereby increasing the likelihood of identifying optimal or near-optimal patterns. Using weights instead of assigning equal probabilities to all items is an optimization that allows prioritizing items with higher frequency, as they are more likely to contribute more to compression.

4.1.2. The matrix structure

The HMP-SA and HMP-HC algorithms both employ a matrix structure $\mathbb{M}$ for storing the support of every distinct item pair in the input database D. $\mathbb{M}$ is a set of triples of the form (i, j, s), where $i, j \in \mathcal{I}$ are distinct items $(i \neq j)$, and $s = sup(\{i, j\})$ is the support of the pair $\{i, j\}$ in D.

This matrix can be implemented as a triangular matrix (a two dimensional array) where rows and columns represent items and support values are stored in cells. Constructing the triangular matrix $\mathbb{M}$ is an incremental process that is done efficiently by a single scan of the input database. The process is illustrated in Fig. 2(a–f) for the database of Table 2. Initially, all entries of the matrix are set to zero (Fig. 2(a)). Then, the transaction $t_1 = \{a, c, d, e, f\}$ is scanned and the support of all pairs of items within that transaction is increased by 1 in the matrix. The updated matrix after reading t_1 is shown in Fig. 2(b). It indicates that the itemsets $\{a, c\}, \{a, d\}, \{a, e\}, \{a, f\}, \{c, d\}, \{c, e\}, \{c, f\}, \{d, e\}, \{d, f\}$ and $(e, f\}$ have a support of 1. Then, the next transaction $t_2 = \{d, e, f\}$ is read and the counts in the matrix for pairs $\{d, e\}, \{d, f\}$, and $\{e, f\}$ are increased by 1, as depicted in Fig. 2(c). Then, the same process is repeated for the remaining transactions. The states of the matrix after scanning t_3, t_4 and t_5, are shown in Fig. 2(d–f), respectively. The final matrix in Fig. 2(f) indicates the exact support of all pairs of items in the database.

The matrix construction process is completed in linear time with respect to the number of transactions, and is performed only once by HMP-SA and HMP-HC, which makes the cost reasonable. In terms of size, a triangular matrix implemented as a two dimensional array requires $\mathcal{O}(|\mathcal{I}|^2)$ memory, and constant time ($\mathcal{O}(1)$) lookup operations for the support of any item pair. However, a triangular matrix may not be memory-efficient for databases containing a large number of items because the matrix stores all possible item pairs, including those with zero support. Thus, an alternative sparse implementation of the matrix $\mathbb{M}$ is considered, where $\mathbb{M}$ is a hashmap of hashmaps instead, formally defined as:

$$\mathbb{M} = \{(i, H_i) \mid i \in \mathcal{I}, H_i = \{(j, s_{ij}) \mid j \in \mathcal{I}, s_{ij} = sup(\{i, j\}), s_{ij} > 0, i < j\}\}. \tag{16}$$

where i in the outer hashmap corresponds to an item, and its associated value H_i is an inner hashmap that stores only those items j for which the pair $\{i, j\}$ appears at least once in the database, along with their corresponding support values s_{ij}. This representation ensures that only nonzero entries are stored, which can reduce memory consumption for sparse datasets. The sparse matrix for the database of Table 2 is shown in Fig. 3.

In the sparse matrix representation, the space complexity is proportional to the number of item pairs that actually occur in the database, rather than to the total number of possible pairs. Specifically, the space requirement is reduced from $\mathcal{O}(|\mathcal{I}|^2)$ to $\mathcal{O}(|\mathcal{P}|)$, where $|\mathcal{P}|$ denotes the number of distinct item pairs with nonzero support in the database. Moreover, lookup operations can still be performed efficiently in expected $\mathcal{O}(1)$ time due to the constant-time access property of hashmaps. Updating the matrix during the database scan is also efficient: when processing a transaction t, for each unordered pair of items $(i, j) \subseteq t$, the algorithm simply increments the corresponding entry s_{ij} (or creates it if it does not yet exist). This sparse representation is well-suited for datasets where the number of items is high but co-occurrence among items is relatively low. This approach is adopted in the remainder of the paper.

4.2. The HMP-SA algorithm

The first algorithm in the proposed HMP framework is named HMP-SA. It adopts a SA-based approach to iteratively improve candidate itemsets. The inputs of HMP-SA are a transaction dataset D, a maximum code table size $maxCodeTableSize$ (representing the number of itemsets to be output), and the cooling rate $\alpha \in (0, 1)$ to control the optimization process.

(a) Initial state of the matrix $\mathbb{M}$

Item	a	b	c	d	e	f
b	0					
c	0	0				
d	0	0	0			
e	0	0	0	0		
f	0	0	0	0	0	

(b) $\mathbb{M}$ after scanning $t_1 = \{a, c, d, e, f\}$

Item	a	b	c	d	e	f
b	0					
c	**1**	0				
d	**1**	0	**1**			
e	**1**	0	**1**	**1**		
f	**1**	0	**1**	**1**	**1**	

(c) $\mathbb{M}$ after scanning $t_2 = \{d, e, f\}$

Item	a	b	c	d	e	f
b	0					
c	1	0				
d	1	0	1			
e	1	0	1	**2**		
f	1	0	1	**2**	**2**	

(d) $\mathbb{M}$ after scanning $t_3 = \{a, c, e\}$

Item	a	b	c	d	e	f
b	0					
c	**2**	0				
d	1	0	1			
e	**2**	0	**2**	2		
f	1	0	1	2	2	

(e) $\mathbb{M}$ after scanning $t_4 = \{a, f\}$

Item	a	b	c	d	e	f
b	0					
c	2	0				
d	1	0	1			
e	2	0	2	2		
f	**2**	0	1	2	2	

(f) $\mathbb{M}$ after scanning $t_5 = \{b, f\}$

Item	a	b	c	d	e	f
b	0					
c	2	0				
d	1	0	1			
e	2	0	2	2		
f	2	**1**	1	2	2	

Fig. 2. The step-by-step construction of the triangular matrix for the database of Table 2, implemented as a two dimensional array. The matrix is initially empty (a) and then updated after scanning each transaction (b-f), with changes in bold. The final matrix after scanning transaction t_5 is shown in (f).

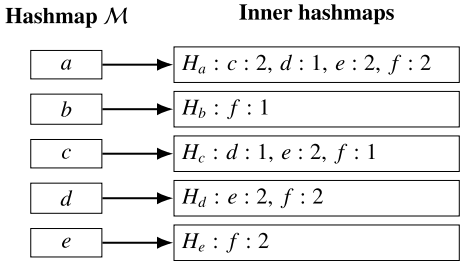


Fig. 3. The sparse matrix $\mathbb{M}$ for the database of Table 2, implemented as a hashmap of hashmaps.

The pseudo-code of HMP-SA is presented in Algorithm 2. The algorithm begins by initializing an empty C to store the discovered itemsets (line 1). Subsequently, it scans the input database D to compute the weights for each item and stores these items along with their weights in a set I_w (line 2). During the same initial database scan, the algorithm also calculates the original size of the database and stores it in a variable S (line 3). Then, the algorithm initializes a counter $maxTry$ to zero, which is used to count unsuccessful pattern generation attempts (line 4). Then, the algorithm enters a primary loop (lines 5 to 31) aimed at iteratively discovering candidate itemsets to enhance the code table C. This loop continues as long as the size of C is less than one-third of the predefined maximum code table size, $maxCodeTableSize$ (line 5), and the algorithm has not exceeded a threshold of unsuccessful attempts to generate a beneficial pattern (line 27). The number of unsuccessful attempts is tracked by the variable $maxTry$. Within the primary loop, the algorithm attempts to identify and add promising itemsets to the code table.

In each iteration of the primary loop, the algorithm first stores the current database size in a variable S_{init} (line 6). Then, the algorithm generates a random pattern p with a length ranging from 3 to Max, where Max is set to 6 (line 7). The generation process for the random pattern is weight-aware, meaning each item's probability of being included in the pattern is proportional to its weight in I_w. Following pattern generation, a SA process is initiated to refine the compression effectiveness of the generated pattern p (lines 8–19). This process begins with an initial temperature $T_{init} = 100$ (line 8) and iteratively reduces the temperature until it falls below a minimum temperature $T_{min} = 0.1$ (line 9). Within each iteration of the SA loop, a neighbor pattern p' is derived from the current pattern p using the `GetNeighbor` method, which either adds a new item to p or removes an item from p (line 10). A temporary C' is then created by tentatively adding this neighbor pattern p' to the current C (line 11). The compression size S' of the database D is calculated using this temporary C' (line 12). The change in compression size, ΔS, is computed as the difference between the current compression size S and the newly calculated compression size S' (line 13).

The algorithm evaluates if the change ΔS is positive, indicating improved compression. If $\Delta S > 0$, the neighbor pattern p' is accepted, becoming the new current pattern p, and the compression size S is updated to S' (lines 14–16). However, even if $\Delta S \leq 0$ (no compression improvement or compression worsened), the neighbor pattern p' may still be accepted probabilistically based on the acceptance criterion: $e^{\Delta S/T} > Random(0, 1)$ (line 14). This probabilistic acceptance, controlled by the current temperature T and a random value, allows the algorithm to escape local optima by occasionally accepting solutions that do not immediately improve compression. After the acceptance check, the temperature T is reduced by multiplying it by the cooling rate α (line

Algorithm 2 HMP-SA.

Input: Database of transactions D, maximum code table size $maxCodeTableSize$, cooling rate α
Output: Optimized code table C

1: $C \leftarrow \emptyset$
2: $I_w \leftarrow$ CalculateItemWeights(D) ▷ Initial weights of items
3: $S \leftarrow$ CalculateDatabaseSize(D)
4: $maxTry \leftarrow 0$ ▷ Counter for unsuccessful pattern generation attempts
5: **while** $|C| < \left\lfloor \frac{1}{3} \times maxCodeTableSize \right\rfloor$ **do**
6: $S_{init} \leftarrow S$
7: $p \leftarrow$GenerateRandomPattern(I_w, 3, Max) ▷ $Max = 6$
8: $T \leftarrow T_{init}$ ▷ $T_{init} = 100$
9: **while** $T > T_{min}$ **do** ▷ $T_{min} = 0.1$
10: $p' \leftarrow$ GetNeighbor(p, I_w, 3, Max)
11: $C' \leftarrow C \cup \{p'\}$
12: $S' \leftarrow$ CalculateCompressionSize(D, C')
13: $\Delta S \leftarrow S - S'$
14: **if** $\Delta S > 0$ or $e^{\Delta S/T} >$ Random(0,1) **then**
15: $p \leftarrow p'$
16: $S \leftarrow S'$
17: **end if**
18: $T \leftarrow \alpha \cdot T$
19: **end while**
20: **if** $S_{init} > S'$ **then**
21: $C \leftarrow C \cup \{p\}$
22: $D \leftarrow$ Delete pattern p from database D
23: $I_w \leftarrow$ CalculateItemWeights(D) ▷ Update weights of items
24: $S_{init} \leftarrow S'$
25: $maxTry \leftarrow 0$
26: **end if**
27: **if** $maxTry > \min(\left\lfloor \frac{1}{3} \times maxCodeTableSize \right\rfloor, 100)$ **then**
28: break
29: **end if**
30: $maxTry \leftarrow maxTry + 1$
31: **end while**
32: $C \leftarrow$ GreedyPatternSelection($D, C, maxCodeTableSize$)
33: **return** C

Algorithm 3 GreedyPatternSelection.

Input: Database D, code table C, maximum code table size $maxCodeTableSize$
Output: An updated code table C

1: **while** $|C| < maxCodeTableSize$ **do**
2: $\mathcal{M} \leftarrow$ CreateMatrix(D) ▷ Dynamically generates the current matrix
3: $(i, j, s) \leftarrow \arg\max_{(i',j',s') \in \mathcal{M}} s'$ ▷ Selects the most frequent 2-itemset
4: $p^* \leftarrow \{i, j\}$
5: $C \leftarrow C \cup \{p^*\}$
6: $D \leftarrow$ Delete pattern p^* from database D
7: **end while**
8: **return** C

18), gradually decreasing the probability of accepting worse patterns as the SA progresses.

Once the SA concludes for the current pattern p (temperature reaches T_{min}), the algorithm assesses whether the best pattern found during annealing (now represented by p) resulted in a compression gain by checking if $S_{init} > S'$ (line 20). If compression improved ($S_{init} > S'$), the pattern p is permanently added to the C (line 21), and occurrences of pattern p are removed from the D (line 22). Following this step, the item weights I_w are recalculated based on the reduced database D to guide subsequent pattern generation (line 23), the current database size S_{init} is set to S' (line 24), and the $maxTry$ counter is reset to 0 (line 25). If the compression did not improve ($\Delta S \leq 0$), the algorithm increments the $maxTry$ counter (line 30).

To prevent indefinite looping in cases where beneficial patterns become scarce, the algorithm includes a check on the $maxTry$ counter (line 27). If $maxTry$ exceeds a limit, set to the minimum between one-third of $maxCodeTableSize$ and 100, the primary pattern generation loop is terminated (lines 27–28).

After exiting the primary loop, the algorithm proceeds to incorporate itemsets of length 2 into the C, as their number is manageable to process comprehensively. This is done by calling the `GreedyPatternSelection` procedure (Algorithm 3) (line 32). This procedure performs a loop to add length-2 patterns in descending order of frequency to further refine compression. The loop continues until the code table size reaches the predefined $maxCodeTableSize$ limit (lines 1 to 7). In the loop, the procedure scans the current, reduced database D to calculate the frequencies of all length-2 itemsets, storing these frequencies in a matrix $\mathcal{M}$ (line 2). Subsequently, the algorithm selects the most frequent length-2 pattern from $\mathcal{M}$ (lines 3 to 4) and adds it to the C (line 5). Then, the pattern is deleted from the database (line 6). The loop stops when the code table contains $maxCodeTableSize$ patterns. These patterns are then returned by the procedure (line 8), and then by the HMP-SA algorithm.

The HMP-SA algorithm relies on two user-defined parameters: the cooling rate α and the maximum code table size $maxCodeTableSize$. The cooling rate α influences the thoroughness of the SA process; a higher α value allows for more extensive exploration of the solution space, potentially leading to more optimal patterns but at the cost of increased runtime due to more iterations per pattern. The number of iterations k in the SA is determined by the formula: $k = \left\lceil \frac{\ln\left(\frac{T_{\min}}{T_{\text{init}}}\right)}{\ln(\alpha)} \right\rceil$. This formula is derived from the iterative cooling process, and in practice, a fixed cooling rate of $\alpha = 0.9$, resulting in 21 iterations, is used as it has been empirically found to offer a balance between runtime and compression effectiveness. The $maxCodeTableSize$ parameter controls the maximum number of patterns in the C. Setting this parameter to values that are either too small or too large can negatively impact the final compression size $L_H(C, D)$. A too small $maxCodeTableSize$ value might prevent the code table from capturing essential patterns, while an excessively large $maxCodeTableSize$ can lead to an increased encoded size because the total encoded size $L_H(D, C)$ is dependent on $L_H(C|D)$ as defined in equation (Eq. 13).

4.2.1. Detailed Example

A detailed example is now presented to illustrate the execution of HMP-SA. Consider that the database of Table 2 is taken as input with $\alpha = 0.8$ and $maxCodeTableSize = 3$.

Initialization: The algorithm first initializes the code table C as empty. Then, the database is scanned to calculate the weights of the items. As explained in Section 4.1.1, the weights for this database are: $I_w = \{(a, 0.6), (b, 0.2), (c, 0.4), (d, 0.4), (e, 0.6), (f, 0.8)\}$. Then, assuming that each integer is encoded using 32 bits, the initial size of the database is calculated as $S = 32 \times 15 = 480$ bits since the database contains 15 items.

Main Loop: Then, the main loop is performed (lines 5 to 31) to add candidate itemsets to the code table until the required code table size is reached.

First Iteration: Assume that a random pattern $p = \{a, c, e, f\}$ is generated and the iterative process based on SA is performed to improve the pattern (lines 9 to 19). The temperature parameter T is set to the initial temperature of 100. A neighboring pattern $p' = \{a, c, e\}$ is generated by removing the item f from p (line 10). Then, the code table is updated as $C' = C \cup \{p'\} = \{\{a, c, e\}\}$ (line 11). Subsequently, the new compression size, denoted as S', is computed. This value depends

on two primary components: the size of the modified database, and the size of the updated code table C'. The size of the code table C' is computed by summing the size of its entries. Here, C' contains one entry containing the itemset $\{a, c, e\}$ and its code. The itemset $\{a, c, e\}$ is encoded using $3 \times 32 = 96$ bits because it contains three integers, and its code size is given by $-\log_2(\frac{2}{5}) = 1.32$ since it appears in two out of five transactions. Thus, the code table size is $96 + 1.32 = 97.32$ bits. The modified database size, is then obtained by replacing $\{a, c, e\}$ with its code $code(\{a, c, e\})$ in the original database, as shown in Table 5. Since the modified database contains 9 items encoded using 32 bits and the code $code(\{a, c, e\})$ appears twice, the modified database size is $9 \times 32 + 2 \times 1.32 = 290.64$ bits. The change in compression size is then computed as $\Delta S = S_{init} - S' = 480 - (290.64 + 97.32) = 92.04$ bits.

Because an improvement in compression is achieved ($\Delta S > 0$), the pattern $\{a, c, e\}$ is kept. Then, the temperature is reduced by updating $T \leftarrow 0.95 \times T$, resulting in $T = 95$ and the iterative pattern improvement process is repeated. This inner loop continues until the temperature T is less than or equal to $T_{min} = 0.1$. Once cooling is complete, the final pattern p is added to C, and the pattern is deleted from D. For this example, consider that the final pattern is $\{a, c, e\}$.

Subsequent Iterations: The algorithm then repeats the aforementioned steps for generating additional patterns until the code table contains a number of patterns equal to one third of $maxCodeTableSize$. In this example, with $C = \{a, c, e\}$ and $maxCodeTableSize = 3$, no more iterations are needed.

Finalizing the Code Table: Then, the algorithm calls the procedure GreedyPatternSelection to fill the code table with patterns of size 2 following an iterative greedy process. During an iteration, the procedure: 1) constructs the frequency matrix $\mathbb{M}$ from the current database, 2) selects the pattern that has the highest frequency, 3) adds the pattern to the code table, and 4) removes all its instances from the database.

For the modified database of Table 5, the most frequent pattern of size 2 is $\{d, f\}$, which is thus added to the code table. Then, the next most frequent pattern of size 2 is $\{a, f\}$ or $\{b, f\}$. For this example, we consider that $\{a, f\}$ is chosen and the resulting code table is $C = \{\{a, c, e\}, \{d, f\}, \{a, f\}\}$. Since this code table has $maxCodeTableSize = 3$ patterns, the algorithm stops and this is the final code table.

The final compression size using this code table is calculated as follows. The compressed database is shown in Table 6. The code table size has three entries for $\{a, c, e\}$, $\{d, f\}$, and $\{a, f\}$. In these entries, the itemsets have 3, 2, and 2 items, respectively, and their code lengths are $-\log_2(\frac{2}{5}) = 1.32$, $-\log_2(\frac{2}{5}) = 1.32$, $-\log_2(\frac{1}{5}) = 2.32$ because they appear 2 times, 2 times and one time, respectively. Thus, the code table size is: (3×32+1.32)+(2×32+1.32)+(2×32+2.32) = 228.96. The modified database contains 3 items and the codes of $\{a, c, e\}$, $\{d, f\}$, and $\{a, f\}$ have 2, 1, and 1 occurrences in the modified database, respectively. Thus, the modified database size is: $(3 \times 32) + (2 \times 1.32) + (2 \times 1.32) + (1 \times 2.32) = 103.6$. The final compression size is then calculated as $228.96 + 102.28 = 332.56$ bits.

Table 5
Transaction database after removing the itemset $\{a, c, e\}$.

Transactions	Items
t_1	$code(\{a, c, e\}), d, f$
t_2	d, e, f
t_3	$code(\{a, c, e\})$
t_4	a, f
t_5	b, f

Table 6
Transaction database after removing the itemsets $\{a, c, e\}$, $\{d, f\}$ and $\{a, f\}$.

Transactions	Items
t_1	$code(\{a, c, e\}), code(\{d, f\})$
t_2	$code(\{d, f\}), e$
t_3	$code(\{a, c, e\})$
t_4	$code(\{a, f\})$
t_5	b, f

4.2.2. Time complexity analysis

The time complexity of Hmp-Sa can be expressed by examining and aggregating the costs of three components: (1) the initialization phase, (2) the SA phase, and (3) the greedy-pattern-selection phase.

In the initialization phase, the algorithm constructs an empty code table in constant time, $\Theta(1)$, then performs a scan of the database to compute item weights and the initial compression size in $\Theta(n)$ time. Hence the total cost of initialization is

$$\Theta(1) + \Theta(n) = \Theta(n).$$

During the SA phase, the outer loop iterates until the code table contains $\lfloor z/3 \rfloor$ patterns, that is, $\Theta(z)$ iterations, where $z = maxCodeTableSize$. In each iteration, the algorithm generates a candidate pattern in $\Theta(m)$ time. Then, an inner loop is executed k times, each iteration producing a neighbor in $\Theta(m)$ time and recomputing the compression by scanning the database in $\Theta(n)$ time. The inner loop therefore costs $m + \Theta\big(k\,(m+n)\big)$. Afterward, the algorithm appends the accepted pattern to the code table in $\Theta(1)$ and then performs two additional database scans—one to remove the pattern and one to update item weights—costing $2\Theta(n) = \Theta(n)$. Thus each outer iteration costs

$$\Theta\big(1 + m + k(m+n) + n\big) = \Theta\big(m + k(m+n) + n\big),$$

and the entire SA phase costs

$$\Theta\big(z\,(m + k(m+n))\big).$$

In the greedy-pattern-selection phase, the algorithm adds the remaining $\lceil 2z/3 \rceil = \Theta(z)$ patterns by, in each iteration, rebuilding the sparse co-occurrence matrix in $\Theta(n)$ time, scanning up to $\Theta(m^2)$ matrix entries to select the most frequent pattern, and then removing that pattern in another $\Theta(n)$ database scan. Each iteration thus costs

$$\Theta\big(n + m^2 + n\big) = \Theta(m^2 + n),$$

and the phase as a whole costs

$$\Theta\big(z\,(m^2 + n)\big).$$

Summing the three components yields the overall time complexity

$$\Theta(n) + \Theta\big(z\,(m + k(m+n))\big) + \Theta\big(z\,(m^2+n)\big) = \Theta\big(n + z[m + k(m+n) + m^2 + n]\big).$$

From this expression, it can be seen that if $m^2 \gg m + k(m+n)$ then the greedy-selection term $\Theta(z\,m^2)$ dominates, whereas if $m + k(m+n) \gg m^2$ the simulated-annealing term $\Theta(z(\,m + k(m+n)))$ dominates. In many practical scenarios n is large relative to m^2, and the matrix is often very sparse rather than containing the maximum of m^2 entries. Besides, the value of k is generally much smaller than m and n (for example, $k = 21$ for the suggested values of T_{min} and T_{init}). Thus the repeated database scans ($\Theta(z\,n)$) govern the running time.

In addition to this complexity analysis, it is worth noting that the deletion of patterns from the database in line 22 of the algorithm contributes to its efficiency. As more patterns are removed from the database and added to the code table, the database progressively becomes smaller, and the cost of scanning the database decreases. This deletion technique was initially introduced in Grimp [32].

4.3. The HMP-HC algorithm

The HMP-HC algorithm takes a transaction database D as input, a maximum code table size $maxCodeTableSize$ (representing the number of itemsets to be output), and a maximum number of generations G_{max}. The algorithm HMP-HC uses a process similar to HMP-SA, where patterns are randomly generated and iteratively improved using a heuristic process. However, while HMP-SA relies on a SA process where the number of iterations depends on the temperature, HMP-HC utilizes a HC approach with a fixed number of generations determined by a hyperparameter G_{max}. Moreover, the main difference between HMP-HC and HMP-SA is that, while the latter includes a probabilistic acceptance criterion that occasionally accepts worse solutions to escape local optima, HMP-HC exclusively accepts better solutions.

Algorithm 4 provides the pseudo-code for HMP-HC. It begins by initializing the code table as empty (line 1), calculating item weights (line 2) and computing the original database size S (line 3). The algorithm then initializes a $maxTry$ counter to 0 (line 4). Thereafter, the algorithm enters a main loop (lines 5 to 31) that continues until the code table size reaches one third of $maxCodeTableSize$ (line 5).

In the main loop, the algorithm first saves the current database size in a variable S_{init} (line 6). Then, a pattern p is randomly generated with a length between 3 to $Max = 6$, inclusive (line 7). Then, a HC loop starts to iteratively improve the pattern p to obtain better compression (lines 9 to 19). A counter G is initialized to zero (line 8) and incremented during each iteration of the HC loop (line 18), and the loop terminated when $G < G_{max}$ (line 9). In the HC loop, the `GetNeighbor` function is first called to refine the pattern p (line 10). The resulting pattern p' is tentatively added to the code table C (line 11). Then, the compressed size of the database is calculated (line 12). If it is improved, i.e. $\Delta S = S - S' > 0$ (lines 13–14), the improved pattern p' is kept (line 14) and the current compressed size is noted (line 15). When the HC loop terminates, if the final pattern p produces any compression improvement (line 19), it is permanently added to the code table C (line 21), and removed from the database (line 22). Moreover, the item weights are recalculated (line 23), the current database size is saved in S_{init} (line 24), and the $maxTry$ counter is reset to 0 (line 25). If the final pattern p does not yield any compression improvement, the $maxTry$ counter is incremented (line 30). If $maxTry$ reaches one third of $maxCodeTableSize$ or 100, it indicates that, despite many attempts, the algorithm has failed to improve the compression, and thus the main loop is stopped (lines 27 to 29).

After the main loop, the algorithm calls the procedure `GreedyPatternSelection` to fill the code table up to $maxCodeTableSize$ with itemsets of length 2, drawn in descending order of frequency (line 32). This is done as in HMP-SA and thus the process is not described again here. Finally, the algorithm returns the final code table (line 33).

Similar to other MDL-based compression approaches, HMP-HC guarantees that each refinement step preserves or improves the compression ratio, ultimately producing a code table C that delivers an optimized encoding of the database. This monotonic property, together with user control over the generation limit $G_{\max}$, offers a balanced approach, which allows to trade computational effort for improved encoding performance.

Algorithm 4 HMP-HC.

Input: Database of transactions D, maximum code table size $maxCodeTableSize$, and number of generations G_{max}
Output: Optimized code table C and final compression size S

1: $C \leftarrow \emptyset$
2: $I_w \leftarrow$ CalculateItemWeights(D) ▷ Initial weights of items
3: $S \leftarrow$ CalculateDatabaseSize(D)
4: $maxTry \leftarrow 0$ ▷ Counter for unsuccessful pattern generation attempts
5: **while** $|C| < \left\lfloor \frac{1}{3} \times \text{maxCodeTableSize} \right\rfloor$ **do**
6: $S_{init} \leftarrow S$
7: $p \leftarrow$GenerateRandomPattern($I_w, 3, Max$) ▷ $Max = 6$
8: $G = 0$;
9: **while** $G < G_{max}$ **do**
10: $p' \leftarrow$ GetNeighbor(p, I_w, 3, Max)
11: $C' \leftarrow C \cup \{p'\}$
12: $S' \leftarrow$ CalculateCompressionSize(D, C')
13: $\Delta S \leftarrow S - S'$
14: **if** $\Delta S > 0$ **then**
15: $p \leftarrow p'$
16: $S \leftarrow S'$
17: **end if**
18: $G \leftarrow G + 1$
19: **end while**
20: **if** $S_{init} > S'$ **then**
21: $C \leftarrow C \cup \{p\}$
22: $D \leftarrow$ Delete pattern p from database D
23: $I_w \leftarrow$ CalculateItemWeights(D) ▷ Updated weights of items
24: $S_{init} \leftarrow S'$
25: $maxTry \leftarrow 0$
26: **end if**
27: **if** $maxTry > \min(\left\lfloor \frac{1}{3} \times maxCodeTableSize \right\rfloor, 100)$ **then**
28: break
29: **end if**
30: $maxTry \leftarrow maxTry + 1$
31: **end while**
32: $C \leftarrow$ GreedyPatternSelection($D, C, maxCodeTableSize$)
33: **return** C

4.3.1. Detailed Example

This subsection presents a detailed example of the execution of HMP-HC on the database in Table 2 with $G_{max} = 40$ and $maxCodeTableSize = 3$, assuming that integers are encoded using 32 bits.

Initialization: The code table C is initialized as empty. Item weights are then computed as: $I_w = \{(a, 0.6), (b, 0.2), (c, 0.4), (d, 0.4), (e, 0.6), (f, 0.8)\}$. Since the original database contains 15 items, its size is $S = 32 \times 15 = 480$ bits.

Main Loop: The algorithm enters the main loop to add candidate itemsets to the code table until the required code table size is met.

First Iteration: A random pattern $p = \{a, d, e, f\}$ is first generated. Then, the HC process begins to refine the pattern p, and runs from $G = 0$ until $G < G_{\max}$. Within this loop, a neighboring pattern $p' = \{d, e, f\}$ is produced. The code table is tentatively updated to $C' = C \cup \{p'\} = \{\{d, e, f\}\}$. Following this, the updated compression size, denoted as S', is calculated as the sum of C' and the size of the database compressed using C'. The code table C' contains a single entry for the itemset $\{d, e, f\}$. The itemset $\{d, e, f\}$ is encoded using $3 \times 32 = 96$ bits, while its code size is $-\log_2\left(\frac{2}{5}\right) = 1.32$ bits because it appears in two out of five transactions. Hence, the size of C' is $96 + 1.32 = 97.32$ bits. The modified database obtained by substituting $\{d, e, f\}$ with its code $code(\{d, e, f\})$ is depicted in Table 7. Its size is $9 \times 32 + 2 \times 1.32 = 290.64$ because it contains 9 items encoded using 32 bits, and the code $code(\{d, e, f\})$ occurs twice. The change in compression size is then computed as $S_{init} - S' = 480 - (290.64 + 97.32) = 92.04$ bits, and the pattern is kept since there is an improvement. This HC loop continues to try to refine the pattern until $G = 40$. No better pattern is found, and $\{d, e, f\}$ is permanently added to C.

Subsequent Iterations: The algorithm continues to execute the main loop to add patterns to the code table until it reaches one third of $maxCodeTableSize$. In this example, with $C = \{d, e, f\}$, this condition is reached and no other iterations are needed.

Finalizing the Code Table: The next step is to add length 2 patterns to the code table by descending order of frequency to fill the code table up to $maxCodeTableSize$ patterns. This process is carried out by the `GreedyPatternSelection` procedure, as in HMP-SA. The patterns $\{a, c\}$ is first selected because it appears the most frequently (twice) in the

Table 7
Transaction database after removing the itemset $\{a, c, e\}$.

Transactions	Items
t_1	$a, c, code(\{d, e, f\})$
t_2	$code(\{d, e, f\})$
t_3	a, c, e
t_4	a, f
t_5	b, f

Table 8
HMP algorithms time complexity.

Algorithm	Time Complexity
HMP-SA	$\Theta\big(n + z[m + k(m+n) + m^2 + n]\big)$
HMP-Hc	$\Theta\big(n + z[m + G_{max}(m+n) + m^2 + n]\big)$

modified database, and then the pattern $\{a, f\}$ is selected. The final code table C is $C = \{d, e, f\}, \{a, c\}, \{a, f\}$ and the final compression size S is 332.56 bits.

4.3.2. Time complexity analysis

The time complexity of the HMP-HC algorithm is almost the same as that of HMP-SA since the structure of both algorithms is quite similar, with the main distinction being that HMP-SA performs additional operations related to updating the temperature and acceptance probability and verifying the probabilistic acceptance condition. Furthermore, instead of the parameter α used in HMP-SA to control the number of iterations, HMP-HC uses a fixed parameter, G_{max}, which limits the number of generations and plays a similar role in determining the iteration count. As a result, the complexity remains largely unchanged, with the overall time complexity of HMP-HC being:

$$\Theta\big(n + z[m + G_{max}(m+n) + m^2 + n]\big)$$

Similarly to HMP-SA, the time complexity of HMP-HC scales mainly with the number of transactions (n), the number of distinct items (m), the code table size (z), and the number of generations (G_{max}). The time complexity of the two proposed algorithms is summarized in Table 8.

5. Experimental evaluation

This section presents the experimental analysis of HMP-SA and HMP-HC. The first subsection describes the experimental settings, including competing benchmark methods, computational environment specifications, and dataset characteristics. Subsequent subsections presents the results of systematic evaluations of itemsets quality through compression ratio, runtime and memory usage, convergence behavior, and classification performance.

5.1. Experimental settings

Algorithms: The HMP-SA and HMP-HC algorithms were implemented in *Java*,[1] while the original $C++$ implementations of GRIMP,[2] KRIMP,[3] and SLIM[4] were employed for comparison. These three baselines were selected to ensure a fair and meaningful comparison within the MDL-based itemset mining domain. Although a variety of related algorithms are discussed in Section 2, most target other data types or pattern forms—such as sequential, graph-based, or numerical data—and are therefore not directly comparable. All algorithms were run on the same computing platform to ensure a fair assessment of each algorithm across multiple datasets. It should be noted that the use of Java for HMP-SA and HMP-HC incurs an inherent runtime disadvantage, since Java source code is compiled to bytecode and executed on a virtual machine, whereas C++ is compiled directly to native machine code. For all experiments, the parameters were set to $T_0 = 100$, $T_{min} = 0.1$, $\alpha = 0.9$, and $G_{max} = 40$. To justify these settings, we conducted a sensitivity analysis of the main parameters (T_0, α, and G_{max}) on representative datasets. The results show that HMP's performance is stable across different parameter ranges, with only minor variations observed.

Environment: The experiments were conducted on a system equipped with a 13th-generation Intel Core i5 processor, 32 GB of RAM, and running Windows 11 Professional. Each algorithm was allotted a maximum of two hours to complete processing for each dataset.

Datasets: A total of 21 publicly available benchmark and real datasets were used to evaluate and compare the performance of HMP with the compared methods. These datasets were used in the GRIMP and KRIMP paper, while only a subset was used in the SLIM paper. The datasets have varying characteristics, which ensure that the results are applicable and generalizable in a broad range of real-world use cases. All datasets are structured as transactional data (collections of itemsets), and include several large and dense datasets from the LUCS/KDD repository,[5] the FIMI repository,[6] the UCI repository,[7] and the SPMF library.[8] A summary of these datasets, including their key statistics, is provided in Table 9. In this table, $|D|$ represents the total number of transactions in a dataset, $DI(D)$ is the number of distinct items in the dataset, $ATL(D)$ refers to the average number of items per transaction, and the dataset density is calculated as:

$$Density(D) = \frac{ATL(D)}{DI(D)} \times 100 \tag{17}$$

Table 9
Dataset descriptions and their statistics.

Dataset	$\|D\|$	$DI(D)$	$ATL(D)$	$Density(D)$
Accidents	340,183	468	33.80	7.22
Adult	48,842	97	35.60	15.33
BMSWV1	59,602	497	2.53	0.51
Breast	699	16	9.98	62.37
Chess (k-k)	3,196	75	37	49.33
Chess (kr-k)	28,056	58	7.00	12.07
Connect-4	67,557	129	43	33.33
Heart	303	50	13.98	27.96
Ionosphere	351	157	35.01	22.29
Iris	150	19	5.01	26.32
Led7	3,200	24	7.99	33.33
Letter Recognition	20,000	102	17.17	16.67
Mushroom	8,124	119	23.01	19.33
Nursery	12,960	32	9.01	28.13
Page Blocks	5,473	44	11.00	25
Pen Digits	10,992	86	17.00	19.77
Pima	768	38	9.06	23.68
Pumsbstar	49,046	2,088	50.52	2.42
Retail	88,162	16,470	9.88	0.06
Tic-tac-toe	958	29	9.99	34.48
Wine	178	68	14.00	20.59

5.2. Compression ratio

The first experiment evaluates how well the patterns found by HMP-SA and HMP-HC describe a dataset by comparing their compression ratios. Both HMP-SA and HMP-HC were evaluated in two configurations: one employing the matrix (in which one third of the itemsets are

[1] github.com/EnZ0cez/HMPs
[2] github.com/MuhammadzohaibNawaz/GRIMP
[3] vreeken.eu/prj/krimp/
[4] vreeken.eu/prj/slim/
[5] csc.liv.ac.uk/frans/KDD/Software/LUCS-KDD-DN/
[6] fimi.uantwerpen.be/data/
[7] archive.ics.uci.edu/
[8] philippe-fournier-viger.com/spmf/index.php?link=datasets.php

Table 10
Compression and runtime for all HMP-SA and HMP-HC across all datasets. Each entry in the table indicates the compression achieved (numerator) and the required time to achieve this compression (denominator).

Dataset	Itemset count	HMP-SA		HMP-HC	
		Matrix	No Matrix	Matrix	No Matrix
Accidents	1583	**6.13** / **3144.05**	–	6.35 / 6862.68	–
Adult	1201	2.97 / **21.74**	10.31 / 329.93	**2.81** / 44.72	11.19 / 512.26
BMSWV1	718	51.95 / **7.61**	–	51.95 / 9.71	–
Breast	30	2.80 / **0.008**	1.96 / 0.09	**1.93** / 0.03	3.16 / 0.48
Chess (k-k)	292	**5.93** / **7.42**	13.59 / 35.12	6.18 / 11.99	13.38 / 100.79
Chess (kr-k)	1060	6.00 / 78.96	25.33 / 206.39	**4.49** / 105.07	27.05 / 618.69
Connect-4	1671	**1.72** / **656.65**	16.60 / 3555.06	1.96 / 2121.87	17.42 / 7418.416
Heart	79	16.03 / 0.09	31.21 / 0.09	17.81 / **0.02**	36.59 / 0.29
Ionosphere	170	24.98 / **0.33**	35.13 / 1.50	**24.46** / 0.51	37.81 / 3.52
Iris	13	20.9 / 0.006	27.87 / 0.006	**18.89** / **0.002**	27.66 / 0.01
Led7	152	4.62 / **0.29**	5.09 / 1.13	**3.98** / 0.86	6.06 / 5.92
Letter Recognition	1599	**4.92** / **138.59**	24.86 / 401.48	5.02 / 246.79	24.87 / 1142.80
Mushroom	442	**5.54** / **4.77**	16.53 / 47.64	6.13 / 16.43	17.40 / 101.92
Nursery	260	7.51 / 4.36	12.52 / 32.73	6.56 / 5.98	13.37 / 107.39
Page Blocks	53	**0.39** / **0.08**	1.28 / 0.79	0.44 / 0.41	1.43 / 2.01
Pen Digits	1247	**6.75** / **136.77**	21.98 / 327.26	6.95 / 179.19	23.52 / 1024.8
Pima	58	**6.02** / **0.01**	5.99 / 0.46	6.48 / 0.04	6.82 / 0.61
Pumsbstar	331	15.34 / **196.51**	92.75 / 175.87	**13.39** / 226.906	91.73 / 77.074
Retail	6264	64.36 / 4604.35	–	**64.31** / 4533.07	–
Tic-tac-toe	160	**14.22** / **0.18**	14.75 / 0.18	14.31 / 0.41	18.88 / 2.71
Wine	63	**36.35** / **0.01**	58.14 / 0.06	36.55 / **0.01**	68.45 / 0.12
Average	830.76	15.501 / **7.165**	23.105 / 284.20	**14.33** / 11.402	24.821 / 617.76

generated via the heuristic and two-thirds via the matrix), and one relying exclusively on the heuristic to generate the required itemsets. The algorithms were run 10 times and their best results are reported, following the methodology of GRIMP [32]. For a given database D and a code table C produced by an algorithm, the compression ratio, denoted as $CR\%$, is computed as:

$$CR(D,C) = \frac{L_H(D,C)}{L(D)}\% \tag{18}$$

where $L(D)$ represents the original database size. A lower CR indicates better performance.

The results for the compression ratio and runtime of HMP-SA and HMP-HC are presented in Table 10. The results in the table are presented in the format $\frac{CR}{Time}$: CR in the numerator and computational time, in seconds, in the denominator. For example, the first entry $\frac{6.13}{3144.05}$ for HMP-SA with matrix indicates that it achieved a 6.13 % CR with a runtime of 3,144.05 seconds. Boldface highlights the best observed values for either CR or runtime. A dash (–) indicates that the algorithm did not complete within the two-hour time limit. Excluding excessively long executions helps to keep the focus on solutions that are practical and achievable within acceptable time constraints. From the results, it can be observed that using the matrix generally outperformed not using it in terms of both runtime and CR, often by significant margins–especially on larger datasets such as *Adult, Letter Recognition*, and *Pen Digits*. This suggests that employing the matrix to store and identify frequent itemsets of length 2 contributes to more efficient computation and compression. HMP-SA often achieves faster runtimes, while HMP-HC occasionally attains better compression (14.33 %) compared to HMP-SA (15.501 %), at the cost of longer execution times. On smaller datasets like *Wine, Iris*, and *Pima*, both perform well with minimal runtime differences. This shows that the overhead associated with managing the matrix or heuristic generation has a negligible effect on overall runtime.

Table 11
Comparison of CR and runtime of HMP-SA, HMP-HC, Grimp, KRIMP, and SLIM, with results presented using the format $\frac{\text{CR}}{\text{time}}$.

Dataset	*HMP-SA*	*HMP-HC*	*GRIMP*	*KRIMP*	*SLIM*
Accidents	**6.13** / **3144.05**	6.35 / 6862.68	6.50 / 6408.80	55.10 / –	31.10 / –
Adult	2.97 / **21.74**	**2.81** / 44.72	3.50 / 128.40	24.40 / 205.00	22.80 / 240.45
BMSWV1	51.95 / **7.61**	51.95 / 9.71	**48.40** / 903.40	85.90 / –	84.00 / –
Breast	2.80 / **0.008**	**1.93** / 0.03	22.90 / 0.17	17.00 / 0.39	–
Chess (k-k)	**5.93** / **7.42**	6.18 / 11.99	9.90 / 10.34	30.00 / –	14.70 / 22.70
Chess (kr-k)	4.80 / 105.07	**4.49** / 105.07	11.10 / **63.31**	61.60 / 68.58	57.50 / 86.46
Connect-4	**1.72** / **656.65**	1.96 / 2121.87	3.30 / 1021.80	42.90 / –	12.30 / –
Heart	16.03 / 0.09	17.81 / **0.02**	**12.00** / 0.22	57.70 / 4.81	–
Ionosphere	24.98 / **0.33**	24.46 / 0.51	**24.10** / 1.17	59.80 / ––	49.70 / 65.51
Iris	20.9 / 0.006	**18.89** / **0.002**	29.60 / 0.02	48.20 / 0.06	–
Led7	4.62 / **0.29**	**3.98** / 0.86	4.90 / 3.26	28.60 / 1.20	–
Letter Recognition	4.92 / **138.59**	5.02 / 246.79	**3.00** / 179.54	35.70 / 2796.01	33.40 / –
Mushroom	5.54 / **4.77**	6.13 / 16.43	**5.40** / 30.52	20.50 / –	18.50 / 45.21
Nursery	7.51 / 4.36	6.56 / 5.98	**2.20** / **1.92**	45.50 / 12.16	–
Page Blocks	**0.39** / **0.08**	0.44 / 0.41	2.56 / 1.04	5.00 / 1.32	–
Pen Digits	**6.75** / **136.77**	6.95 / 179.19	18.40 / 56.96	42.20 / 3370.80	39.40 / 1364.87
Pima	**6.02** / **0.01**	6.48 / 0.04	16.70 / 0.44	34.80 / 0.81	–
Pumsbstar	15.34 / **196.51**	**13.39** / 226.906	13.70 / 523.70	56.00 / –	25.10 / –
Retail	64.36 / 4604.35	**64.31** / 4533.07	65.60 / **1099.40**	97.50 / 4031.54	–
Tic-tac-toe	**14.22** / **0.18**	14.31 / 0.41	19.60 / 1.09	56.00 / 2.00	–
Wine	**36.35** / **0.01**	36.55 / **0.01**	44.60 / 0.20	77.40 / 2.01	–

Table 11 presents a comparison of the CR and runtime of HMP-SA and HMP-HC with existing MDL-based methods: Grimp, KRIMP and SLIM. Compared to GRIMP, both HMP variants demonstrate comparable or improved compression quality while significantly reducing runtime across most datasets. For example, on the *Adult* dataset, HMP-SA achieves a competitive compression ratio of 2.97 % versus GRIMP's 3.50 % and completed execution in 21.74 s–nearly six times faster. This performance reflects the simulated annealing approach of HMP-SA, which balances thorough pattern exploration with time efficiency. When compared to the previous exact algorithm, both HMP variants exhibit substantial improvements. However, in pattern-rich datasets like *Retail*, HMP-SA's emphasis on quality increased the runtime to 4,604 seconds–nearly 4 times longer than GRIMP's 1,099. Overall, the proposed algorithms offer better CR and runtime, substantially outperforming KRIMP and SLIM in efficiency and effectiveness.

To assess the statistical significance of the observed performance differences between HMP and the baseline algorithms, we applied the Wilcoxon signed-rank test [18] to the CR and runtime results across all datasets. This non-parametric test is suitable for paired comparisons without assuming normality of the data. When comparing HMP-SA against GRIMP, the test yielded $p < 0.01$ for runtime, indicating a statistically significant advantage of HMP-SA in terms of speed. For compression ratio, the difference was not statistically significant ($p > 0.05$), confirming that HMP-SA achieves its speedup while maintaining comparable compression quality. Similar analyses against KRIMP and SLIM

Table 12
Comparison of code table lengths of HMP-SA and HMP-HC with GRIMP, KRIMP, and SLIM to achieve equivalent CRs, with results presented using the format $\frac{HMP-SA}{HMP-HC}$.

Dataset	KRIMP	SLIM	GRIMP	Reduction (vs GRIMP) (%)
Accidents	$\frac{106}{100}$	$\frac{163}{175}$	$\frac{1529}{1538}$	$\frac{3.41}{2.84}$
Adult	$\frac{93}{81}$	$\frac{96}{98}$	$\frac{799}{576}$	$\frac{33.47}{52.03}$
BMSWV1	$\frac{27}{27}$	$\frac{34}{34}$	$\frac{1089}{1089}$	$\frac{-51.67}{-51.67}$
Breast	$\frac{18}{20}$	–	$\frac{11}{11}$	$\frac{63.33}{63.33}$
Chess (k-k)	$\frac{63}{63}$	$\frac{145}{125}$	$\frac{194}{204}$	$\frac{33.56}{30.13}$
Chess (kr-k)	$\frac{115}{113}$	$\frac{125}{123}$	$\frac{674}{579}$	$\frac{36.41}{45.37}$
Connect-4	$\frac{63}{41}$	$\frac{249}{255}$	$\frac{896}{886}$	$\frac{46.37}{46.97}$
Heart	$\frac{13}{13}$	–	–	–
Ionosphere	$\frac{47}{45}$	$\frac{62}{61}$	$\frac{162}{164}$	$\frac{4.71}{3.52}$
Iris	$\frac{7}{9}$	–	$\frac{11}{11}$	$\frac{15.38}{15.38}$
Led7	$\frac{21}{22}$	–	$\frac{97}{145}$	$\frac{36.18}{4.61}$
Letter Recognition	$\frac{114}{110}$	$\frac{114}{116}$	$\frac{2517}{2681}$	$\frac{-57.41}{-67.67}$
Mushroom	$\frac{145}{145}$	$\frac{138}{144}$	$\frac{459}{578}$	$\frac{-3.85}{-30.77}$
Nursery	$\frac{29}{30}$	–	–	–
Page Blocks	$\frac{9}{10}$	–	$\frac{10}{10}$	$\frac{81.13}{81.13}$
Pen Digits	$\frac{259}{224}$	$\frac{237}{238}$	$\frac{396}{389}$	$\frac{68.24}{68.88}$
Pima	$\frac{12}{15}$	–	$\frac{23}{22}$	$\frac{60.34}{62.07}$
Pumsbstar	$\frac{39}{46}$	$\frac{146}{147}$	$\frac{342}{635}$	$\frac{-3.32}{-91.84}$
Retail	$\frac{1}{1}$	–	$\frac{5331}{5409}$	$\frac{14.89}{13.65}$
Tic-tac-toe	$\frac{42}{43}$	–	$\frac{117}{107}$	$\frac{26.88}{33.13}$
Wine	$\frac{21}{24}$	–	$\frac{46}{49}$	$\frac{26.98}{22.22}$
Average	$\frac{59.23}{56.28}$	$\frac{137.18}{137.81}$	$\frac{773.84}{793.84}$	$\frac{22.90}{15.96}$

showed statistically significant improvements of HMP in both runtime and compression ratio ($p < 0.01$).

To further evaluate the effectiveness of HMP, we report in Table 12, for each dataset, the number of patterns that HMP-SA and HMP-HC require to achieve the same compression ratios as the three baselines (KRIMP, SLIM, and GRIMP). The first column lists the datasets. The second column shows the number of patterns for HMP-SA and HMP-HC, formatted as $\frac{Hmp-Sa}{Hmp-Hc}$, required to match the compression ratio of KRIMP. The third and fourth columns present the same information but for matching the compression ratios of SLIM and GRIMP, respectively. Finally, the last column indicates the percentage reduction in the number of patterns used by HMP-SA and HMP-HC compared to GRIMP to achieve the same compression ratio. For example, for the *Accidents* dataset, the value $\frac{3.41}{2.84}$ of the last column means that HMP-SA and HMP-HC use 3.41 % and 2.84 % fewer patterns, respectively, than GRIMP. Negative values in the last column indicate cases where HMP-SA or HMP-HC required more patterns than GRIMP to achieve the same compression ratio. On average across all datasets, HMP-SA and HMP-HC achieve reductions of 22.90 % and 15.96 %, respectively, in the number of patterns relative to GRIMP to achieve similar levels of data compression. It is also observed that HMP-SA and HMP-HC require considerably fewer patterns than KRIMP and SLIM on average.

5.3. Runtime and memory usage

For computational efficiency, as presented in Table 10, HMP-SA consistently demonstrated lower computation time than HMP-HC in both

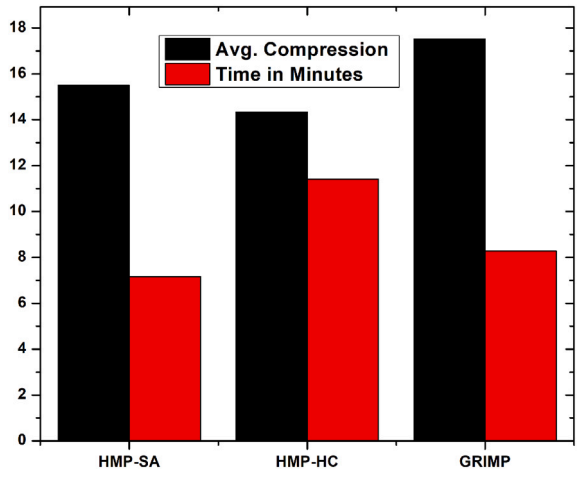


Fig. 4. Comparison of average CR and runtime of HMP-SA, HMP-HC and GRIMP.

the matrix and no-matrix configurations. Specifically, in the no-matrix (matrix) setting, HMP-SA is approximately 1.59 (2.17) times faster than HMP-HC. The average runtimes, measured in minutes, for HMP-SA, HMP-HC, and GRIMP are depicted in Fig. 4, with the results for HMP-SA and HMP-HC reflecting the matrix-based configuration. Among the three approaches, HMP-SA achieved the fastest execution times, followed by GRIMP, with HMP-HC being the slowest.

Additionally, Fig. 5 further analyzes the runtime performance as a percentage of the total observed runtime, including not only HMP-SA and GRIMP but also KRIMP and SLIM. Overall, HMP-SA consistently demonstrates superior speed compared to GRIMP, KRIMP and SLIM, with the exception of the *Chess (kr-k)* and *Retail* datasets, where it performs less efficiently, and on the *Pen Digits* dataset, where it trails slightly behind GRIMP. It is important to note that some baseline algorithms do not have reported runtimes for some datasets in this comparison as they failed to complete within the two-hour limit. For instance, on the *Accidents*, *BMSWV1*, *Connect-4*, and *Pumsbstar* datasets, KRIMP and SLIM have no results.

The effect of different matrix representations on memory consumption for HMP-SA and HMP-HC is evaluated with three configurations (Table 13): (i) no matrix, (ii) triangular matrix, and (iii) sparse matrix. Without using any matrix, both algorithms consume the least memory since only the code table and candidate patterns are maintained. However, this setting is less time-efficient, as all patterns are then generated through the heuristic process. Between the two heuristics, HMP-HC generally uses less memory (average of 172.6 MB across 21 datasets) than HMP-SA (average of 235.1 MB across 21 datasets) in this configuration, which can be attributed to the simpler greedy acceptance criterion of HC compared to the probabilistic acceptance criterion of SA. The triangular matrix provides constant-time access to pairwise supports, but at the expense of much higher memory consumption. For HMP-SA and (HMP-HC), the triangular matrix uses about 39.2 % (64.4 %) more memory than the no-matrix configuration. Although it improves runtime, the overhead becomes prohibitive as the number of items grows.

The sparse matrix offers the best trade-off. By storing only nonzero entries, it achieves memory consumption close to the no-matrix configuration while still enabling efficient support lookups. For HMP-SA (HMP-HC), the sparse matrix variant uses about 19.4 % (63.2 %) more memory than the no-matrix configuration. In contrast, the sparse matrix variant requires about 14.2 % (0.7 %) less memory than the triangular matrix for HMP-SA (HMP-HC). This trend confirms that the sparse design scales effectively with larger and sparser datasets. In summary, the no-matrix approach is the most memory-efficient but is slower in practice, while the triangular approach is the fastest but is more

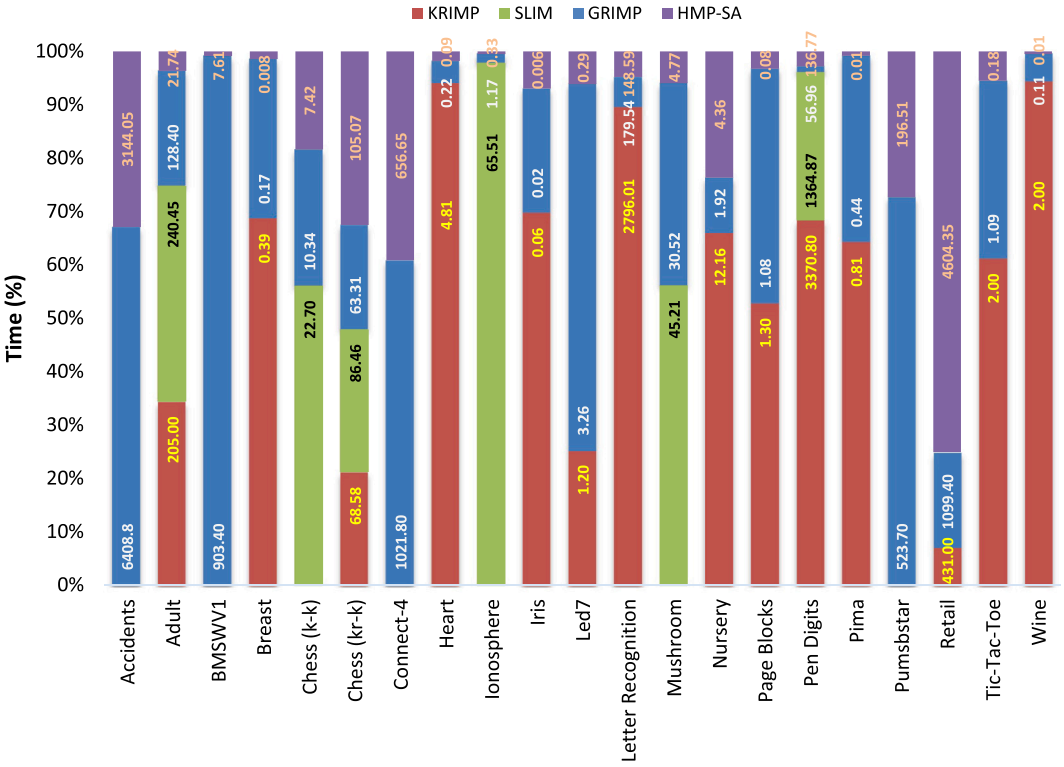


Fig. 5. Comparison of runtime for HMP-SA, GRIMP, KRIMP, and SLIM.

Table 13
Maximum memory usage (MB) comparison of HMP-SA (SA) and HMP-HC (HC), implemented without matrix (NM), with sparse matrix (SM), and with triangular matrix (TM). The *Initial Size* values are expressed in KB.

Dataset	Initial Size (KB)	SA + SM	HC + SM	SA + NM	HC + NM	SA + TM	HC + TM
Accidents	35,009.8	198.6	236.7	125.7	122.8	148.7	240.7
Adult	1,754.2	151.9	169.2	198.5	245.2	165.7	135.9
BMSWV1	935.0	170.9	193.9	98.1	139.5	400.3	579.9
Breast	15.9	277.2	231.5	352.9	107.1	172.6	140.1
Chess(k-k)	319.1	47.3	42.4	72.1	25.1	44.5	22.5
Chess(kr-k)	533.6	132.1	152.4	121.5	71.4	269.0	167.4
Connect-4	7,934.5	126.5	109.9	157.8	158.4	265.7	260.1
Heart	12.0	284.4	266.0	216.7	82.5	269.4	266.3
Ionosphere	34.6	364.1	181.1	210.4	64.0	208.2	236.2
Iris	1.8	26.7	38.8	39.2	29.7	295.5	291.0
Led7	59.5	227.2	367.9	260.8	240.3	295.5	291.0
Letter Recognition	881.5	87.0	75.3	58.0	41.5	77.8	42.8
Mushroom	494.0	340.6	377.9	255.6	156.5	154.0	177.5
Nursery	308.0	237.9	208.2	198.4	184.1	247.9	153.4
Page Blocks	128.8	380.2	487.4	421.6	450.9	179.3	212.9
Pen Digits	511.7	161.8	150.6	125.7	120.9	174.3	234.5
Pima	14.9	581.1	609.8	501.1	252.8	186.9	242.5
Pumbstar	11,027.3	909.9	880.9	445.3	211.4	343.4	375.9
Retail	4,071.2	748.8	616.1	786.8	757.6	1468.2	929.8
Tic-tac-toe	24.8	428.8	511.7	280.7	132.9	1489.6	939.1
Wine	7.2	9.78	8.71	10.97	30.67	15.12	15.12
Average	3051.4	280.6	281.7	235.1	172.6	327.2	283.6

memory-intensive. The sparse implementation balances both aspects, delivering substantial memory savings without compromising the runtime benefits observed in earlier experiments.

5.4. Practical case study

This section presents a practical case study of applying the proposed HMP-SA algorithm to crime records data.[9] This dataset contains 2,662,309 transactions and 35 distinct items with an average transaction length of 1.795 items, and a density of 5.13 %. The goal is to obtain a compressed representation of the database by identifying frequently co-occurring crime types that best describe the data under the MDL principle. In this experiment, we set $maxCodeTableSize = 30$ to identify a small set of highly representative patterns, and use 40 iterations to generate patterns. The resulting code table effectively compresses millions of individual crime records into a concise set of co-occurrence patterns, thereby reducing redundancy while retaining the most informative associations. The obtained compressed patterns (Table 14) highlight

[9] kaggle.com/datasets/currie32/crimes-in-chicago

Table 14
Representative itemsets and their contribution to database compression in the crime dataset.

Itemsets	Support	CR (%)	Itemsets	Support	CR (%)
THEFT, NARCOTICS, CRIMINAL DAMAGE	31,146	98.04	THEFT, NARCOTICS	43,958	71.27
THEFT, OTHER OFFENSE, DECEPTIVE PRACTICE	11,445	97.41	THEFT, BURGLARY	35,562	69.78
OTHER OFFENSE, BATTERY, NARCOTICS	28,144	96.02	OTHER OFFENSE, BATTERY	35,075	68.31
MOTOR VEHICLE THEFT, NARCOTICS, ASSAULT	7,963	95.68	THEFT, ASSAULT	30,004	67.06
THEFT, DECEPTIVE PRACTICE, BATTERY	27,765	94.44	NARCOTICS, CRIMINAL DAMAGE	29,782	65.81
BATTERY, BURGLARY, CRIMINAL DAMAGE, ROBBERY	3,581	94.24	BATTERY, BURGLARY	26,782	64.69
THEFT, BATTERY, CRIMINAL DAMAGE	60,784	92.01	BURGLARY, CRIMINAL DAMAGE	26,326	63.59
OTHER OFFENSE, MOTOR VEHICLE THEFT, ASSAULT	5,774	91.76	THEFT, MOTOR VEHICLE THEFT	25,714	62.52
THEFT, BATTERY, NARCOTICS, ROBBERY	11,174	91.42	NARCOTICS, ASSAULT	23,958	61.51
THEFT, MOTOR VEHICLE THEFT, BATTERY, ASSAULT	6,993	91.28	THEFT, OTHER OFFENSE	22,582	60.57
THEFT, BATTERY	133,239	85.71	OTHER OFFENSE, ASSAULT	21,525	59.67
BATTERY, CRIMINAL DAMAGE	105,092	81.31	THEFT, DECEPTIVE PRACTICE	20,832	58.80
THEFT, CRIMINAL DAMAGE	82,077	77.88	MOTOR VEHICLE THEFT, BATTERY	19,887	57.97
BATTERY, NARCOTICS	65,179	75.15	CRIMINAL DAMAGE, ASSAULT	18,218	57.20
BATTERY, ASSAULT	48,951	73.11	OTHER OFFENSE, CRIMINAL DAMAGE	17,679	56.46

the most common and meaningful correlations among crime types, providing a solid foundation for subsequent analytical and predictive tasks.

The support column indicates the number of transactions (crime records) in which each itemset appears, reflecting how frequently the pattern occurs in the dataset. The CR column denotes the cumulative compression ratio, that is, the proportion of the database size that remains uncompressed when the corresponding pattern is added to the code table. For instance, the top-ranked pattern {THEFT, NARCOTICS, CRIMINAL DAMAGE}, that occurs 31,146 times, yields a CR of 98.04 %, meaning that the encoded database is 98.04 % of the original size, or in other words that 1.96 % of the database is explained by this pattern. Adding the second pattern {THEFT, OTHER OFFENSE, DECEPTIVE PRACTICE}, with a support of 11,445, further reduces the compression ratio to 97.41 %. As more patterns are added, the cumulative compression ratio gradually decreases, indicating that subsequent itemsets generally contribute less to the overall compression. These results demonstrate that a relatively small set of high-support, highly representative itemsets can effectively capture a substantial part (43.53 %) of the database structure, while additional patterns yield only marginal improvements. By balancing support and compression, HMP-SA identifies the most informative co-occurrences of crime types, yielding a concise yet representative summary of the database.

5.5. Convergence

To evaluate the convergence behavior of the HMP-SA and HMP-HC algorithms, additional experiments were conducted to assess how each algorithm's iterative refinement of patterns improves compression efficiency over time. The convergence analysis considered two aspects: (1) the cumulative number of patterns added to the code table C, and (2) the control parameter governing the optimization process—the temperature for HMP-SA versus iteration count for HMP-HC. The convergence results are presented in Fig. 6(a) for HMP-SA and Fig. 6(b) for HMP-HC. Both figures share a common layout: the bottom x-axis shows the cumulative number of patterns added to the code table C, while the upper x-axis represents the respective optimization parameter specific to each method. For HMP-SA, this corresponds to the annealing temperature (decreasing from 100 to 0), while for HMP-HC, it denotes the iteration count (up to a maximum of 30 iterations) used to refine a pattern before adding it to the code table. The y-axis measures the CR, providing a quantitative assessment of the compression efficiency.

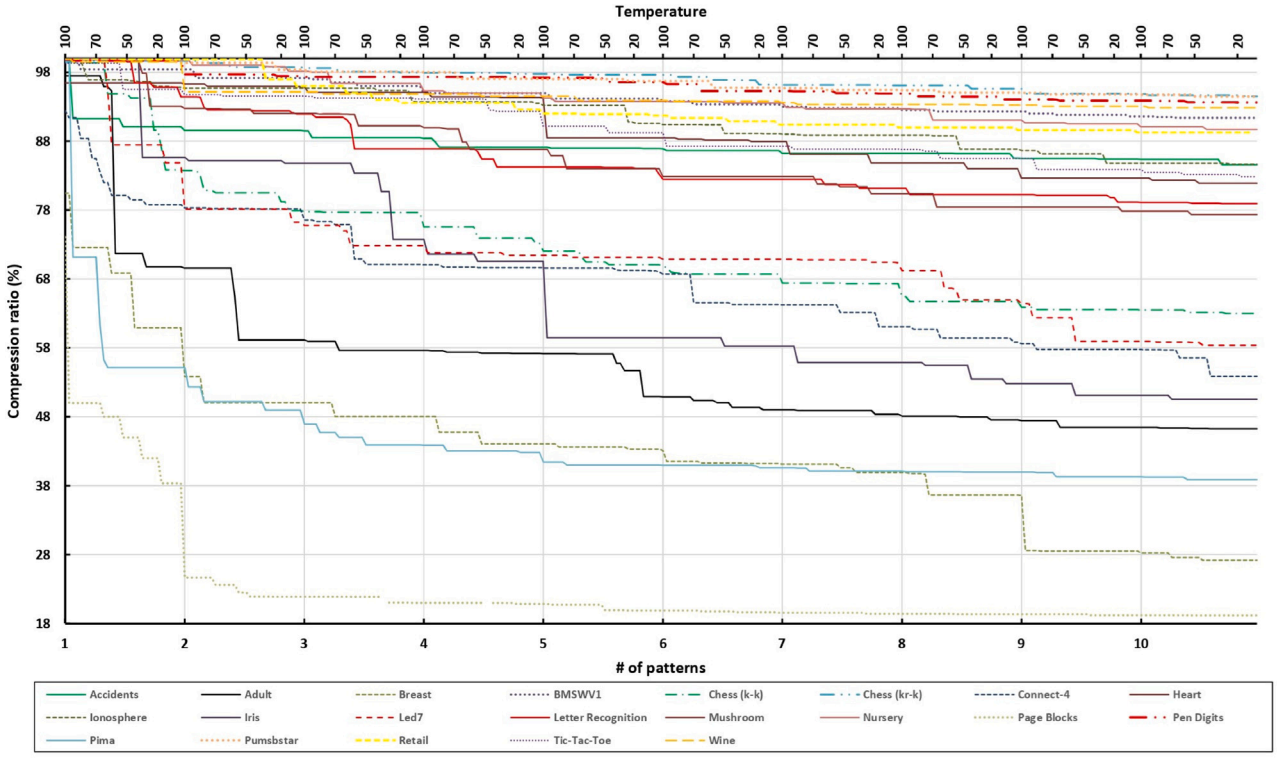
The convergence behavior of HMP-SA, illustrated in Fig. 6(a) demonstrates progressive refinement of compression through the iterative pattern improvement process. On the *Iris* dataset, the compression ratio starts relatively high but drops sharply as the first patterns are added to the code table, indicating rapid early gains before stabilization at larger pattern counts. In contrast, on the *Letter Recognition* and *Mushroom* datasets, HMP-SA exhibits a more gradual decline in CR, reflecting a slower improvement in compression. For *Ionosphere*, HMP-SA demonstrates significant gains from early pattern additions, but improvements stabilize quickly, particularly from patterns 3 to 8, as the dataset's small size requires only a few patterns for effective compression. On the *Pen Digits* dataset, the algorithm shows an irregular trend, pausing at certain iterations before experiencing minor improvements in compression efficiency. A similar trend is observed for the *Chess(kr-k)* dataset—HMP-SA can go several steps without improving the compression ratio, then suddenly reduce it when a new pattern proves beneficial. These variations occur because of the randomness of the heuristic search of HMP-SA: it can become temporarily trapped in a local optimum, making no progress until a new pattern leads to further compression.

Fig. 6(b) depicts the convergence behavior of HMP-HC, driven by its hill-climbing mechanism. On the *Chess (kr-k)* dataset, the compression ratio improves steadily with each added pattern, reflecting consistent gains. Similar trends are observed on *Pen Digits* and *Wine*, with each new pattern contributing modest but reliable compression improvements. In contrast, on the *Page Blocks* dataset, an initial steep improvement is observed during the first pattern's refinement followed by rapid stabilization, suggesting that the most compressible structure is captured immediately. For the *Breast* and *Pima* datasets, steep declines are observed as patterns are added, indicating that hill-climbing is particularly effective on these datasets. These varied convergence profiles illustrate how HMP-HC's optimization process behaves differently with datasets of differing compressibility and structural complexity.

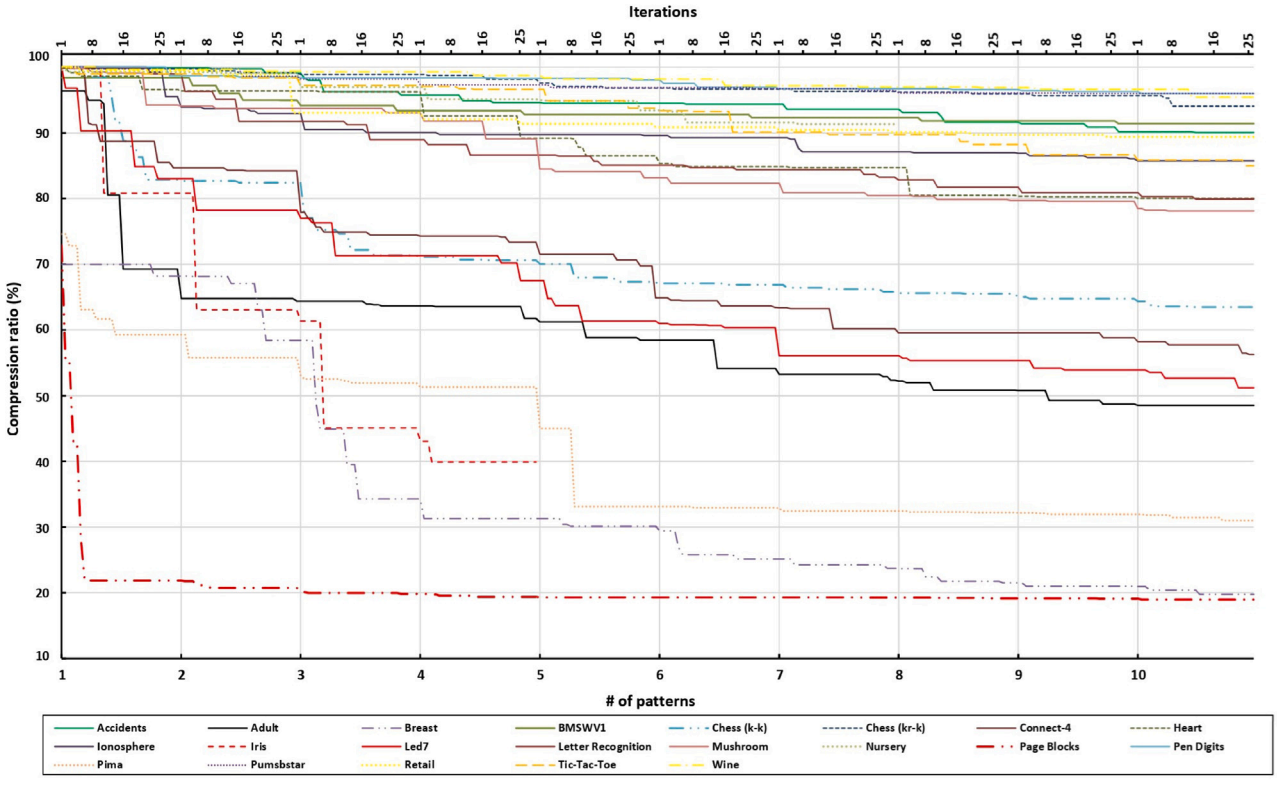
As explained in Section 4, HMP variants are designed so that the optimization process enforces a monotonic non-degradation property: each iteration either preserves or improves the compression ratio. This property is achieved through each algorithm's distinct heuristic approach: temperature-guided exploration in HMP-SA and iteration-constrained refinement in HMP-HC. Fig. 6(a) and (b) clearly demonstrate this principle through two main behavioral patterns across all datasets: (1) steady compression improvement as new patterns are selected, followed by (2) a stable compression with minimal gains.

5.6. Classification

A classification experiment was also conducted to evaluate the discriminative power of the patterns identified by HMP, beyond their demonstrated compression performance. Although HMP is not specifically designed for classification, this experiment provides an assessment of how well the extracted patterns represent the underlying data distributions. For this experiment, we compared classification performance using only the HMP-SA variant—as it outperformed HMP-HC—against GRIMP, KRIMP, and SLIM on the eight datasets that contain class labels and have published results for all competing algorithms.



(a) HMP-SA



(b) HMP-HC

Fig. 6. Comparison of *CRs* ratios with respect to the number of discovered patterns and generations.

Table 15
Comparison of classification accuracy (%).

Dataset	HMP-SA	GRIMP	KRIMP	SLIM
Adult	75	78	79	**80**
Chess (k-k)	**91**	87	89	88
Chess (kr-k)	50	54	50	**56**
Connect-4	**73**	71	68	69
Ionosphere	94	**95**	91	88
Letter Recognition	**75**	73	72	71
Mushroom	93	94	**100**	**100**
Pen Digits	93	**95**	94	93
Average	80.50	**80.88**	80.38	80.63

The methodology uses a custom classifier consistent with the approach used for evaluating GRIMP, KRIMP, and SLIM in their respective papers, ensuring the evaluation is independent of any specific algorithm. Consider a dataset $\mathcal{D} = \{t_1, t_2, \ldots, t_n\}$ where each transaction has a label from a set of labels $\mathcal{L} = \{\ell_1, \ell_2, \ldots, \ell_k\}$. The set of all transactions with a given label $\ell \in \mathcal{L}$ is denoted as $\mathcal{D}_\ell = \{t \in \mathcal{D} \mid t \text{ has label } \ell\}$. For each class $\ell \in \mathcal{L}$, the evaluated algorithm is applied independently to $\mathcal{D}_\ell$ to generate a code table C_ℓ representing that class. Then, for an unseen transaction $t' \subseteq \mathcal{I}$, classification is performed by:

$$\hat{\ell}(t') = \arg\min_{\ell \in \mathcal{L}} L(t' \mid C_\ell), \tag{19}$$

where $L(t' \mid C_\ell)$ denotes the encoded length of t' using code table C_ℓ, following the MDL principle. To estimate classification accuracy, the dataset $\mathcal{D}$ is partitioned into 10 folds $\mathcal{D} = \bigcup_{j=1}^{10} \mathcal{D}^{(j)}$, and 10-fold cross-validation is performed. For each fold j, the code tables $C_\ell^{(-j)}$ are trained on $\mathcal{D} \setminus \mathcal{D}^{(j)}$ and evaluated on $\mathcal{D}^{(j)}$. The accuracy is then computed as:

$$\text{Accuracy} = \frac{1}{n} \sum_{t \in \mathcal{D}} \mathbf{1}\{\hat{\ell}(t) = \ell(t)\}, \tag{20}$$

where $\ell(t)$ is the true class label of transaction t and $\mathbf{1}\{\cdot\}$ is the indicator function.

The accuracy results for HMP-SA and the baselines are presented in Table 15. Overall, the code tables generated by HMP-SA yield high accuracy, demonstrating that the extracted patterns capture key data features. No single algorithm dominates across all eight datasets: HMP-SA and SLIM each achieve the highest accuracy on three datasets, GRIMP on two, and KRIMP on one. By average accuracy, HMP-SA ranks second, closely trailing GRIMP, and then followed by SLIM and KRIMP. In instances where HMP-SA did not achieve better accuracy than GRIMP, such as on the *Ionosphere*, *Mushroom* and *Pen Digits* datasets, HMP-SA's performance remains very close. On overall, these results are considered excellent given that HMP-SA primary design objective is to find patterns by optimizing data compression, while the classification task is used as a secondary way of evaluating the quality and representativeness of the patterns discovered.

Results on the *Mushroom* dataset provides an interesting case study: while KRIMP and SLIM achieved perfect accuracy, HMP-SA accuracy was 93 %. This difference likely arise from HMP-SA's stronger emphasis on compression efficiency, which may result in the exclusion of subtle discriminative information in favor of a more compact and generalized representation of the data. Nevertheless, the overall classification results demonstrate that HMP-SA, despite its focus on optimizing compression, generates code tables that capture meaningful and sufficiently discriminative patterns from the data.

The experimental results presented in this paper collectively demonstrate that HMP-SA and HMP-HC offer significant advantages over GRIMP, KRIMP, and SLIM in terms of compression ratio and runtime. A key finding is HMP's ability to achieve comparable compression ratios while using approximately 22.90 % fewer patterns for HMP-SA and 15.96 % fewer for HMP-HC than competing methods, highlighting its superior efficiency in pattern representation. This representational efficiency confers substantial benefits in large-scale data environments where processing throughput is important. Furthermore, the convergence analysis confirmed HMP's consistent and reliable progression towards optimal compression ratios. Although primarily designed to discover patterns that optimize data compression, HMP-SA exhibited competitive performance in classification tasks, indicating that the resulting code tables retain substantial discriminative power and are suitable for tasks beyond data compression. A $C++$ implementation of the HMP framework is also available[10] to facilitate reproducibility and fair runtime comparison. The results reported in this paper were obtained using the *Java* implementation due to the authors' expertise. Additional experiments on representative datasets using the $C++$ version produced consistent CRs ratios and runtimes, thereby confirming the consistency of our results across implementations.

6. Conclusion

This article introduced HMP, a heuristic framework for pattern mining designed to address the pattern explosion and redundancy issues inherent in traditional frequent itemset mining. By combining the MDL principle with heuristic optimization strategies, HMP enables the discovery of compact and representative pattern sets without requiring exhaustive enumeration. The HMP framework is instantiated by two algorithms: HMP-SA, based on simulated annealing, and HMP-HC, based on hill climbing. A key component of HMP is its binary co-occurrence matrix, which is built by using a single database scan, and stores the support of all distinct item pairs. The matrix allows the fast evaluation of all item pairs without additional database scans, and thus substantially reduces computational cost and improves convergence speed.

Extensive experiments on multiple benchmark datasets demonstrate that both HMP-SA and HMP-HC surpass existing MDL-based approaches such as KRIMP, SLIM, and GRIMP in terms of runtime efficiency and compression effectiveness, while achieving competitive performance in classification. The results highlight the effectiveness of the proposed methods in achieving a balance between computational efficiency and the quality of the mined patterns.

Future work will focus on (1) devising adaptive parameter-tuning mechanisms informed by dataset characteristics; (2) investigating hybrid heuristic strategies that combine SA with HC, or designing adaptive neighborhood mechanisms to improve convergence behavior and overcome stagnation observed in some datasets; (3) developing parallel and distributed implementations of the HMP framework to address large-scale data; (5) extending the approach to other data modalities, including data with utility information [44], graphs [19] and data streams [15]; and (6) integrating HMP outputs with specialized classifiers for anomaly detection to further substantiate the framework's versatility in diverse knowledge-discovery applications.

CRediT authorship contribution statement

Enze Chen: Writing – original draft, Validation, Software, Investigation, Formal analysis, Data curation, Conceptualization. **M. Zohaib Nawaz:** Writing –- review & editing, Validation, Data curation, Conceptualization. **M. Saqib Nawaz:** Writing –- review & editing, Supervision, Methodology, Conceptualization. **Philippe Fournier-Viger:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Investigation, Conceptualization.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

[10] github.com/EnZ0cez/HMPs-Cpp

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The datasets used are publicly available and the access links are provided in the manuscript. The Java code of the HMP framework and the associated datasets are accessible at: github.com/EnZ0cez/HMPs.

References

[1] C.C. Aggarwal, Applications of frequent pattern mining, in: C.C. Aggarwal, J. Han (Eds.), Frequent Pattern Mining, Cham. Springer International Publishing, 2014, pp. 443–467.

[2] R. Agrawal, T. Imieliński, A. Swami, Mining association rules between sets of items in large databases, in: Proccedings of 19th International Conference on Management of Data (SIGMOD'93), 1993, pp. 207–216.

[3] K. Rajwar, K. Deep, S. Das, An exhaustive review of the metaheuristic algorithms for search and optimization: taxonomy, applications, and open challenges, Artif. Intell. Rev. 56 (2023) 13187–13257.

[4] L. Akoglu, H. Tong, J. Vreeken, C. Faloutsos, Fast and reliable anomaly detection in categorical data, in: Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM'12), 2012, pp. 415–424.

[5] M.-B. Belaid, N. Lazaar, Frequent itemset mining with multiple minimum supports: a constraint-based approach, arXiv preprint abs/2109.07844 (2021).

[6] D. Bertsimas, J. Tsitsiklis, Simulated annealing, Stat. Sci. 8 (1) (1993) 10–15.

[7] F. Bonchi, F. Giannotti, D. Pedreschi, A relational query primitive for constraint-based pattern mining, in: J.-F. Boulicaut, L. De Raedt, H. Mannila (Eds.), Constraint-Based Mining and Inductive Databases, Springer, 2006, pp. 14–37.

[8] J.-F. Boulicaut, A. Bykowski, C. Rigotti, Free-sets: a condensed representation of boolean data for the approximation of frequency queries, Data Min. Knowl. Discov. 7 (1) (2003) 5–22.

[9] K. Budhathoki, J. Vreeken, The difference and the norm — characterising similarities and differences between databases, in: Proceedings of European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD 2015), 2015, pp. 206–223.

[10] T. Calders, B. Goethals, Non-derivable itemset mining, Data Min. Knowl. Discov. 14 (1) (2007) 171–206.

[11] M. Faas, M. van Leeuwen, Vouw: geometric pattern mining using the mdl principle, in: Proceedings of 18th International Symposium on Intelligent Data Analysis (IDA 2020), 2020, pp. 158–170.

[12] P. Fournier-Viger, X. Li, J. Yao, J.C.-W. Lin, Interactive discovery of statistically significant itemsets, in: Proceedings of 31st International Conference on Industrial Engineering and Other Applications of Applied Intelligent Systems (IEA/AIE 2018), 2018, pp. 101–113.

[13] P.D. Grünwald, The Minimum Description Length Principle, MIT Press, 2007.

[14] M. Han, Z. Gao, A. Li, S. Liu, D. Mu, An overview of high utility itemsets mining methods based on intelligent optimization algorithms, Knowl. Inf. Syst. 64 (11) (2022) 2945–2984.

[15] M. Han, F. He, R. Zhang, C. Li, F. Meng, High utility itemset mining in data stream using elephant herding optimization, Knowl. Inf. Syst. 67 (3) (2025) 2357–2394.

[16] S. Hess, N. Piatkowski, K. Morik, Shrimp: descriptive patterns in a tree, in: Proceedings of 16th Workshops on Learning, Knowledge, Adaptation, LWA 2014: Knowledge Discovery, Data Mining and Machine Learning, KDML 2014, Information Retrieval, IR 2014 and Knowledge Management, FGWM 2014, 2014, pp. 181–192.

[17] J.H. Holland, Adaptation in Natural and Artificial Systems, MIT Press, 1975.

[18] M. Hollander, D.A. Wolfe, E. Chicken, Nonparametric Statistical Methods, third ed., John Wiley & Sons, Hoboken, NJ, 2013.

[19] L. Hu, Y. Lin, L. Zou, M.T. Özsu, A graph pattern mining framework for large graphs on GPU, The VLDB J. 34 (1) (2025) 1–24.

[20] H.T. Lam, F. Morchen, D. Fradkin, T. Calders, Mining compressing sequential patterns, Stat. Anal. Data Min. 7 (1) (2014) 34–52.

[21] M. Li, P. Vitányi, An Introduction to Kolmogorov Complexity and Its Applications, vol. 4, Springer, 2019.

[22] Y. Li, C. Huang, M. Cheng, T. Lv, Y. Zhao, Y. Sun, Y. Yuan, Privminer: a similar-first approach to frequent itemset mining under local differential privacy, World Wide Web 28 (2) (2025) 25.

[23] P.J. Liang, A. Wang, L. Akoglu, C. Faloutsos, Pattern recognition and anomaly detection in bookkeeping data, Asian Bur. Finance Econ. Res. Work. Pap. (2021).

[24] J. Liu, P. Fournier-Viger, M. Zhou, G. He, M. Nouioua, Cspm: discovering compressing stars in attributed graphs, Inf. Sci. 611 (2022) 126–158.

[25] J.M. Luna, P. Fournier-Viger, S. Ventura, Frequent itemset mining: a 25 years review, WIREs Data Min. Knowl. Discov. 9 (6) (2019) e1329.

[26] T. Makhalova, S.O. Kuznetsov, A. Napoli, Mint: mdl-based approach for mining interesting numerical pattern sets, Data Min. Knowl. Discov. 36 (1) (2022) 108–145.

[27] A. Mantuan, L. Fernandes, Spatial contextualization for closed itemset mining, in: Proceedings of International Conference on Data Mining (ICDM 2018), 2018, pp. 1176–1181.

[28] X. Mi, The mining algorithm of maximum frequent itemsets based on frequent pattern tree, Comput. Intell. Neurosci. 2022 (1) (2022) 7022168.

[29] M.S. Nawaz, P. Fournier-Viger, U. Yun, Y. Wu, W. Song, Mining high utility itemsets with hill climbing and simulated annealing, ACM Trans. Manag. Inf. Syst. 13 (1) (2021a) 1–22.

[30] M.S. Nawaz, M.Z. Nawaz, O. Hasan, P. Fournier-Viger, M. Sun, An evolutionary/heuristic-based proof searching framework for interactive theorem prover, Appl. Soft Comput. 104 (2021b) 107200.

[31] M.S. Nawaz, M.Z. Nawaz, O. Hasan, P. Fournier-Viger, M. Sun, Proof searching and prediction in hol4 with evolutionary/heuristic and deep learning techniques, Appl. Intell. 51 (2021c) 1580–1601.

[32] M.Z. Nawaz, S. Nawaz, P. Fournier-Viger, N. Selmaoui-Folcher, Grimp: a genetic algorithm for compression-based descriptive pattern mining, Expert Systems 42 (5) (2025) e70033.

[33] L. Ni, W. Luo, N. Lu, W. Zhu, Mining the local dependency itemset in a products network, ACM Trans. Manag. Inf. Syst. 11 (1) (2020) 1–31.

[34] N. Pasquier, Y. Bastide, R. Taouil, L. Lakhal, Discovering frequent closed itemsets for association rules, in: Proceedings of 7th International Conference on Database Theory (ICDT'99), 1999, pp. 398–416.

[35] O. Sampson, M.R. Berthold, Widened krimp: better performance through diverse parallelism, in: Proceedings of 13th International Symposium on Intelligent Data Analysis (IDA 2014), vol. 8819, Springer, 2014, pp. 241–252.

[36] B. Selman, C.P. Gomes, Hill-climbing search, Encyclopedia of Cognitive Science 81 (333–335) (2006) 10.

[37] K. Smets, J. Vreeken, Slim: directly mining descriptive patterns, in: Proceedings of 12th SIAM International Conference on Data Mining (SDM 2012), 2012, pp. 236–247.

[38] N. Tatti, J. Vreeken, Finding good itemsets by packing data, in: Proceedings of 8th International Conference on Data Mining (ICDM 2008), 2008, pp. 588–597.

[39] N. Vanetik, M. Litvak, Drim: mdl-based approach for fast diverse summarization, in: Proccedings of 17th International Conference on Web Intelligence (WI 2018), 2018, pp. 660–663.

[40] M.N. Vlashejerdi, N. Daneshpour, Efficient mining of closed high-utility itemsets in dynamic and incremental databases, Eng. Appl. Artif. Intell. 144 (2025) 110081.

[41] J. Vreeken, M. Van Leeuwen, A. Siebes, Krimp: mining itemsets that compress, Data Min. Knowl. Discov. 23 (1) (2011) 169–214.

[42] J. Witteveen, W. Duivesteijn, A. Knobbe, P. Grünwald, Realkrimp - finding hyperintervals that compress with mdl for real-valued data, in: Proceedings of 13th International Symposium on Intelligent Data Analysis (IDA 2014), vol. 8819, Springer, 2014, pp. 321–332.

[43] Y. Yan, L. Cao, S. Madden, E.A. Rundensteiner, Swift: mining representative patterns from large event streams, Proc. VLDB Endow. 12 (3) (2018) 265–277.

[44] Y. Yan, X. Niu, Z. Zhang, P. Fournier-Viger, L. Ye, F. Min, Efficient high utility itemset mining without the join operation, Inf. Sci. 681 (2024) 121218.

[45] D. Zhang, Y. Zhang, Q. Niu, X. Qiu, Mining concise patterns on graph-connected itemsets, Neurocomputing 336 (2019) 27–35.